COSC264
Introduction to Computer Networks and the Internet

Transport Layer Protocols

Dr Barry Wu

WRC
University of Canterbury
Barry.wu@canterbury.ac.nz

Learning Objectives

- At the end of the lectures, you will be able to:
  - Describe transport-layer services
  - Explain the concept of multiplexing and demultiplexing
  - Present an overview of the UDP services
  - Explain the TCP connection management
  - Understand how TCP provides reliable data transfer, flow control, and congestion control

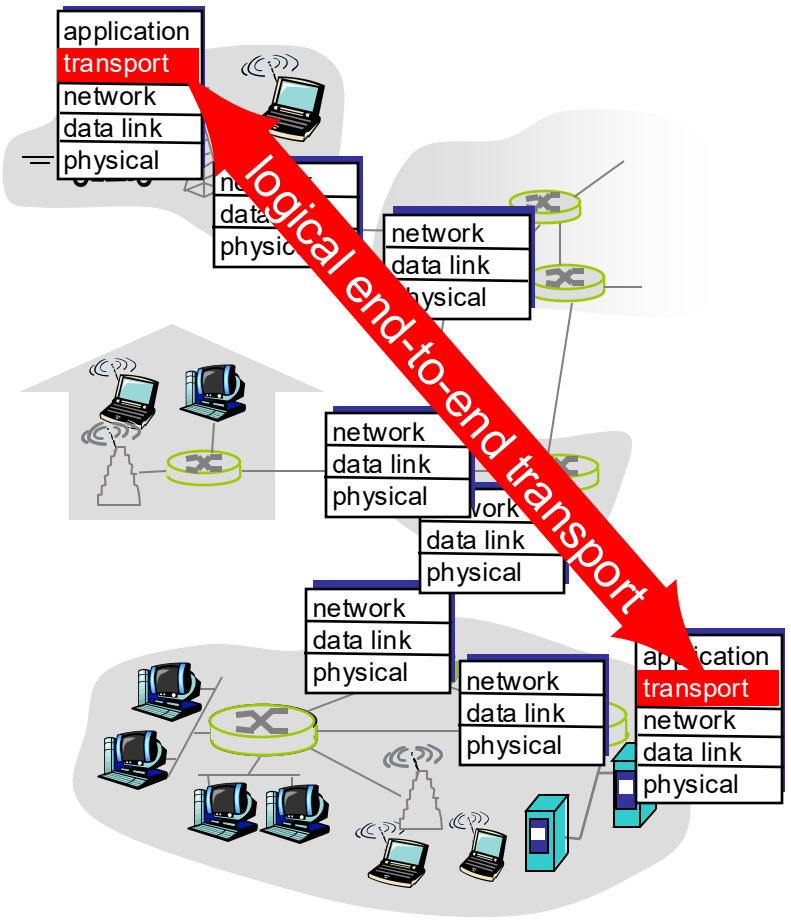
Outline

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Connection-oriented transport: TCP

Introduction to transport-layer services

Transport Protocols

- Provide logical communication between application processes running on different hosts
- Run on end hosts
  - Sender: the transport layer converts the message from a sending application into transport-layer segments and passes them to the network layer
  - Receiver: the transport layer receives segments from below and processes them, making available the data in the segments to the receiving application
- Multiple transport protocols available to applications
  - Internet: mainly TCP and UDP



Services provided by the Internet Transport Layer

- Integrity/error checking (TCP and UDP)
- Extending host-to-host delivery to process-to-process delivery (TCP and UDP);
  - Transport-layer multiplexing and demultiplexing
- Reliable data transfer (TCP)
- Flow control (TCP)
- Congestion control (TCP)

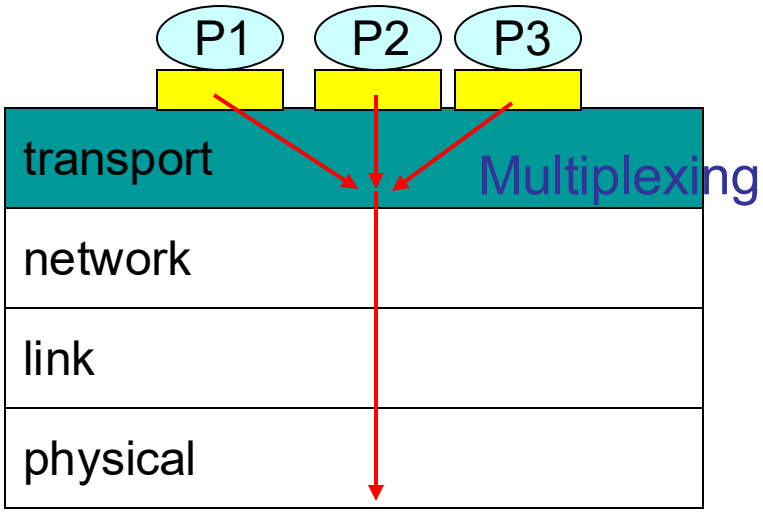
Popular Internet applications and their underlying transport protocols

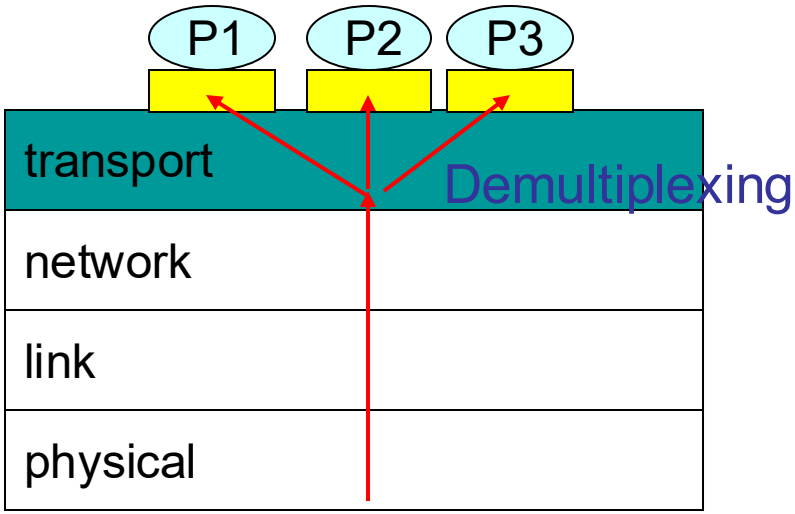
Application	Application-layer protocol	Underlying transport protocol
Email	SMTP	TCP
Web	HTTP	Typically TCP
File transfer	FTP	TCP
Name translation	DNS	Typically UDP
Network management	SNMP	Typically UDP
Streaming multimedia	Typically proprietary	UDP or TCP
Internet telephony	Typically proprietary	UDP or TCP

Multiplexing and demultiplexing

Definition

- Multiplexing
  - To create segments and pass them to the network layer
- Demultiplexing
  - To deliver the data in a transport-layer segment to the correct socket



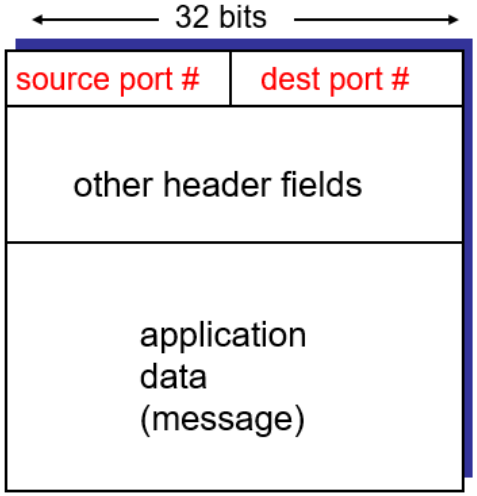


Process

Socket

How demultiplexing works

- The host receives IP datagrams
  - Each datagram has source and destination IP addresses
  - Each datagram carries 1 transport-layer segment
  - Each segment has source and destination port numbers



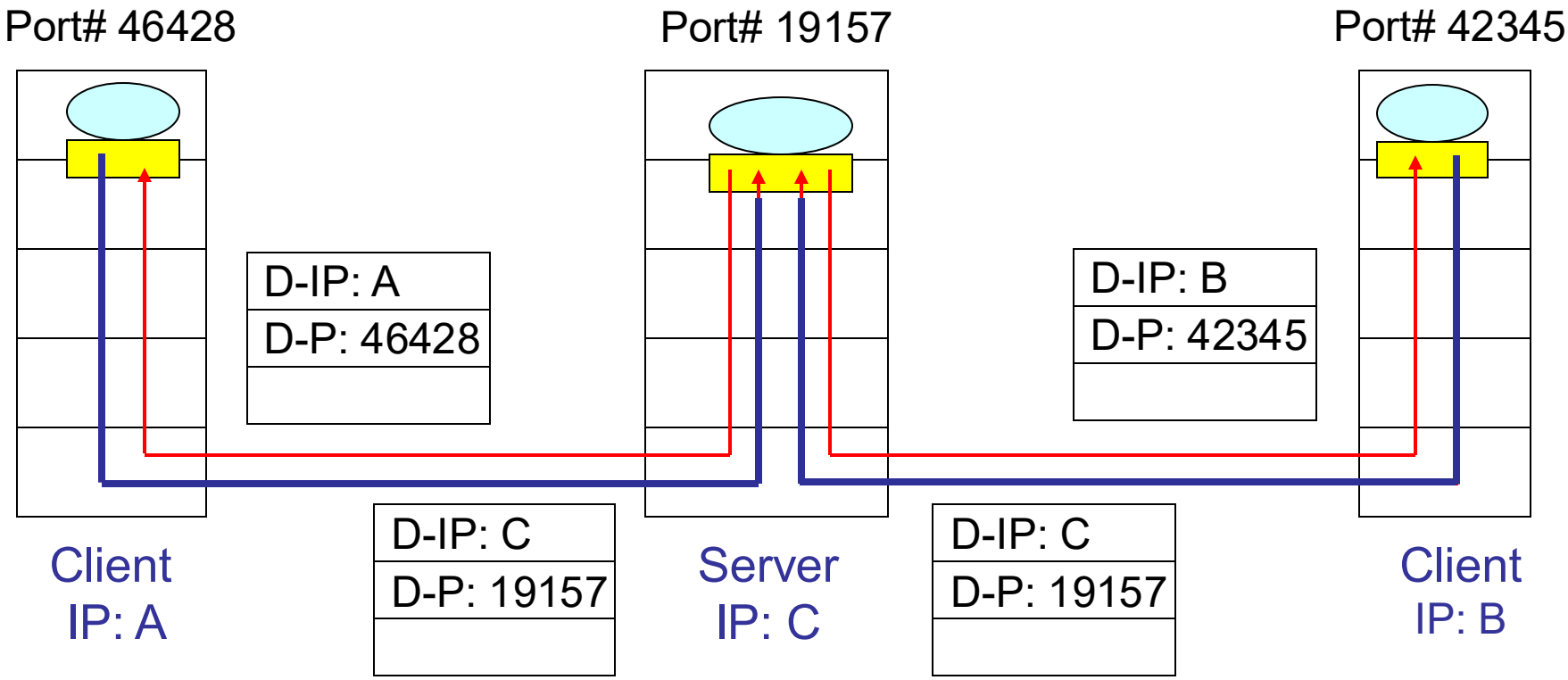
TCP/UDP segment format

- The host uses IP addresses & port numbers to direct a segment to the appropriate socket

Connectionless Demultiplexing

- A UDP socket is identified by a two-tuple
  - Destination IP address
  - Destination port number
- When a host receives a UDP segment
  - checks the destination port number in the segment
  - directs the UDP segment to the socket with that port number
- Packets with different source IP addresses and/or source port numbers, but with the same destination IP and port number, will be directed to the same socket.

Connectionless Demultiplexing (cont.)

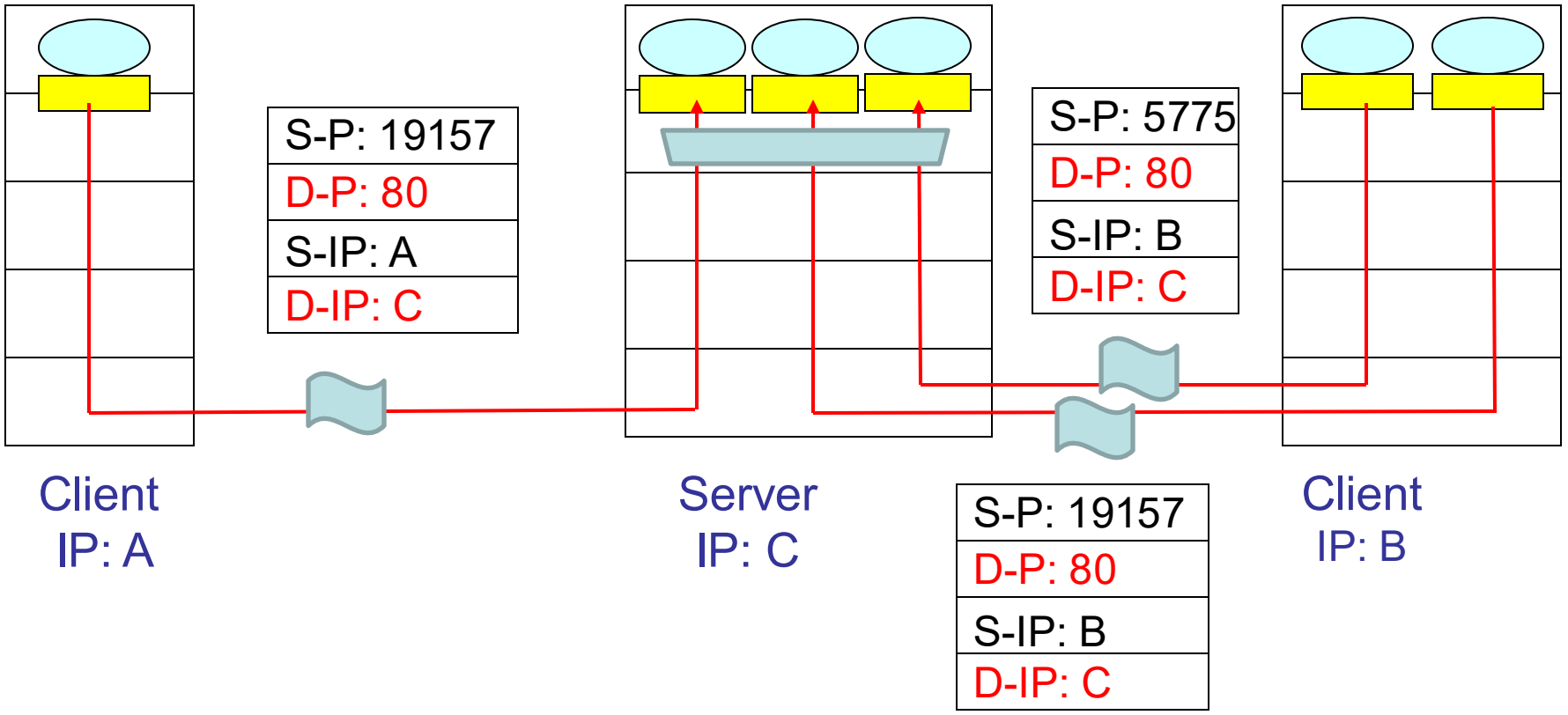


The source port serves as part of a "return address".

Connection-Oriented Demultiplexing

- A TCP socket is identified by a 4-tuple
  - Source IP address
  - Source port number
  - Destination IP address
  - Destination port number
- The server host may support many simultaneous TCP sockets
  - Each socket is attached to a process
  - Each socket is identified by its own 4-tuple

Connection-Oriented Demultiplexing (cont.)



The same Web server has different sockets for different processes at clients.

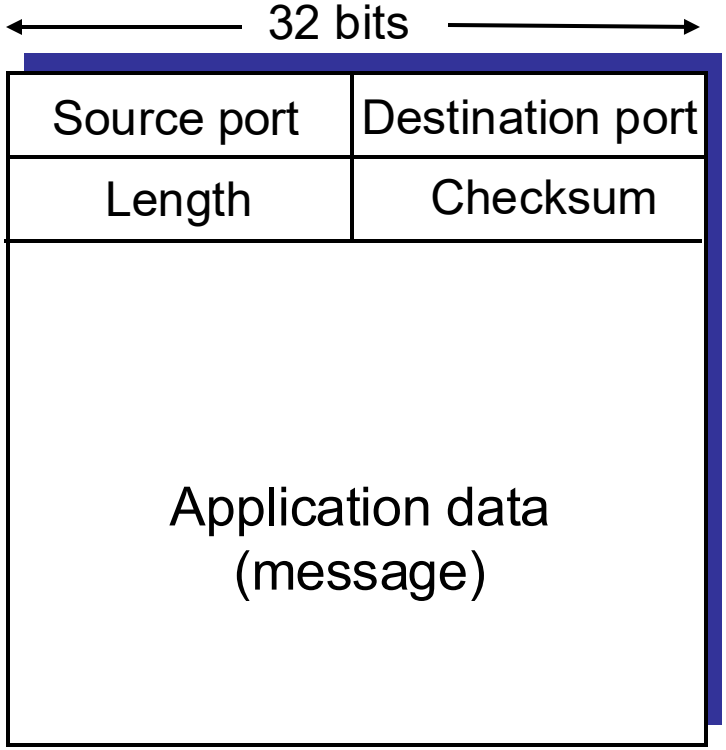
Connectionless transport: UDP

UDP (User Datagram Protocol) [RFC 768]

- A "no-frills, bare-bones" transport protocol
  - Multiplexing/demultiplexing function and error checking
- Provide a connectionless service for application-level processes
  - No handshaking between sending and receiving transport-layer entities before sending a segment
- An unreliable service
  - UDP segments may be lost or delivered out of order

UDP Segment Format

- The source and destination ports add a port addressing capability to IP.
- The length field contains the length of the entire UDP segment, including header and data.
- An optional checksum applies to the entire UDP segment plus a conceptual IP pseudoheader.



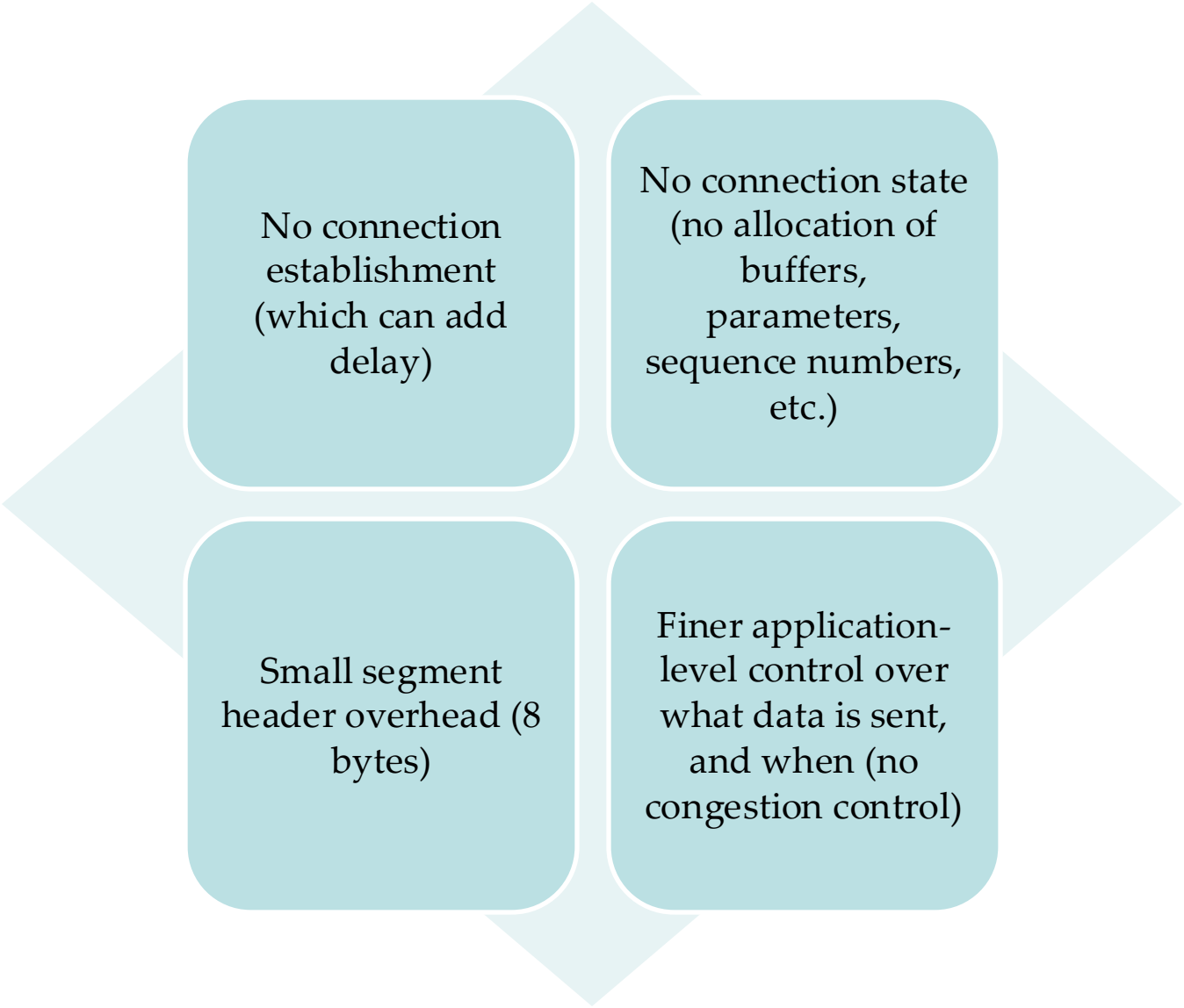
UDP segment format

A UDP segment captured by Wireshark

12	0.508950	10.34.40.169	132.181.2.225	DNS
13	0.509041	132.181.2.225	10.34.40.169	DNS
14	0.509462	10.34.40.169	132.181.2.225	DNS
15	0.510280	132.181.2.225	10.34.40.169	DNS
16	0.510354	132.181.2.225	10.34.40.169	DNS
17	33.503303	10.34.40.169	132.181.2.225	DNS
18	33.504666	132.181.2.225	10.34.40.169	DNS
19	61.002465	10.34.40.169	132.181.2.225	DNS
20	61.004045	132.181.2.225	10.34.40.169	DNS
21	61.013117	10.34.40.169	132.181.2.225	DNS
22	61.014578	132.181.2.225	10.34.40.169	DNS

```
▷ Frame 12: 88 bytes on wire (704 bits), 88 bytes captured (704 bits) on interface 0
▷ Ethernet II, Src: IntelCor_b6:fe:63 (80:19:34:b6:fe:63), Dst: JuniperN_ef:61:00 (2c:21:31:ef:61:00)
▷ Internet Protocol Version 4, Src: 10.34.40.169, Dst: 132.181.2.225
◢ User Datagram Protocol, Src Port: 63507, Dst Port: 53
    Source Port: 63507
    Destination Port: 53
    Length: 54
    Checksum: 0xe576 [unverified]
    [Checksum Status: Unverified]
    [Stream index: 6]
▷ Domain Name System (query)
```

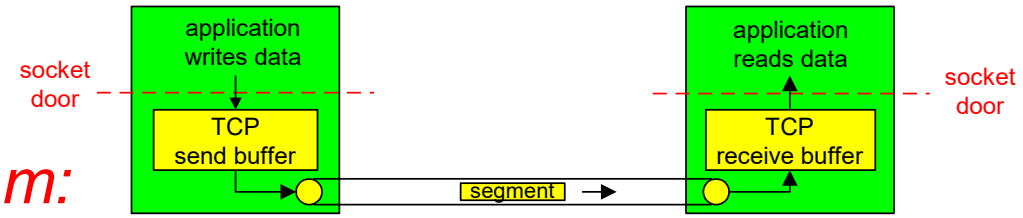
Why using UDP



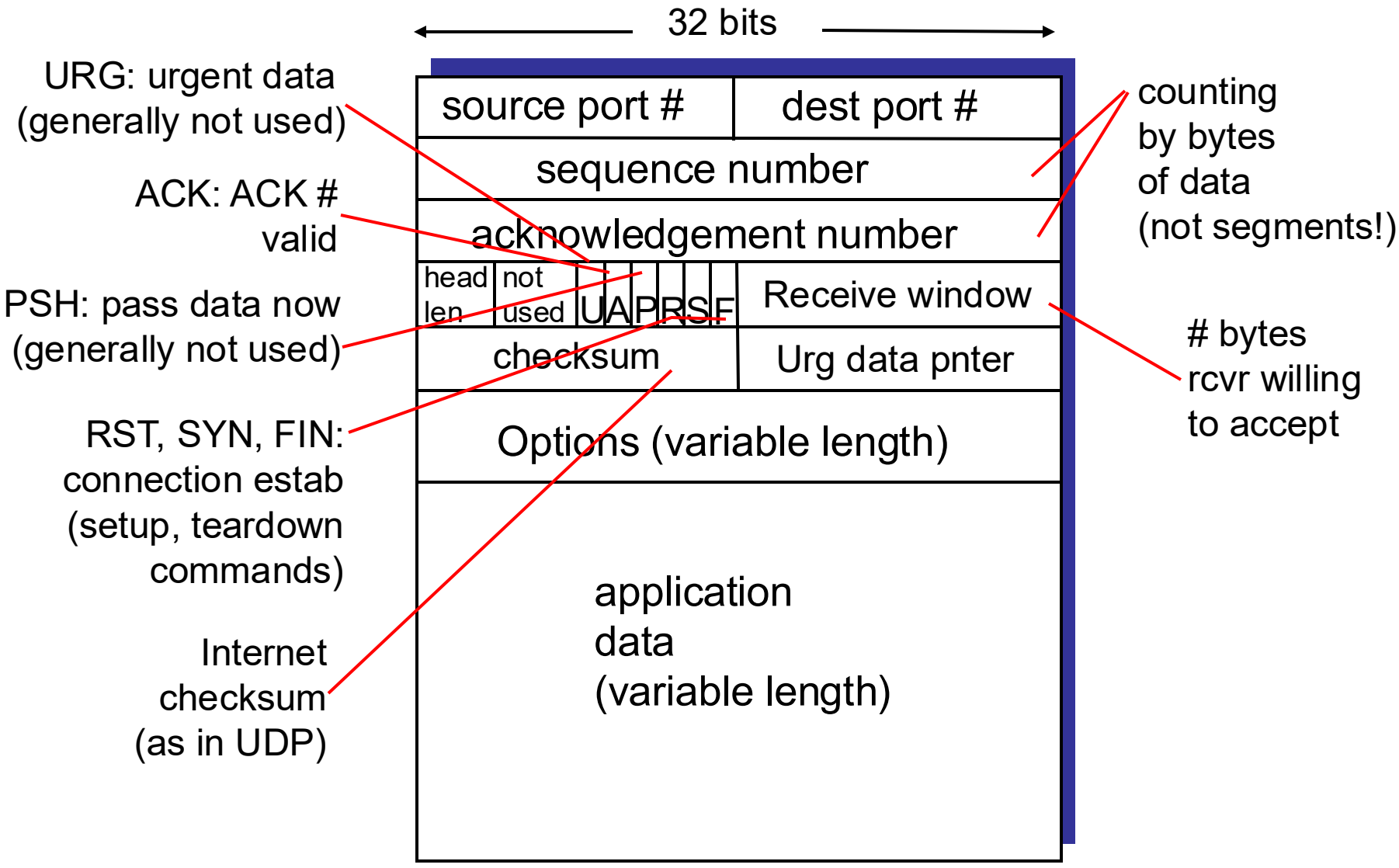
Connection-oriented transport: TCP

TCP: Overview RFCs: 793, 1122, 1323, 2018, 2581

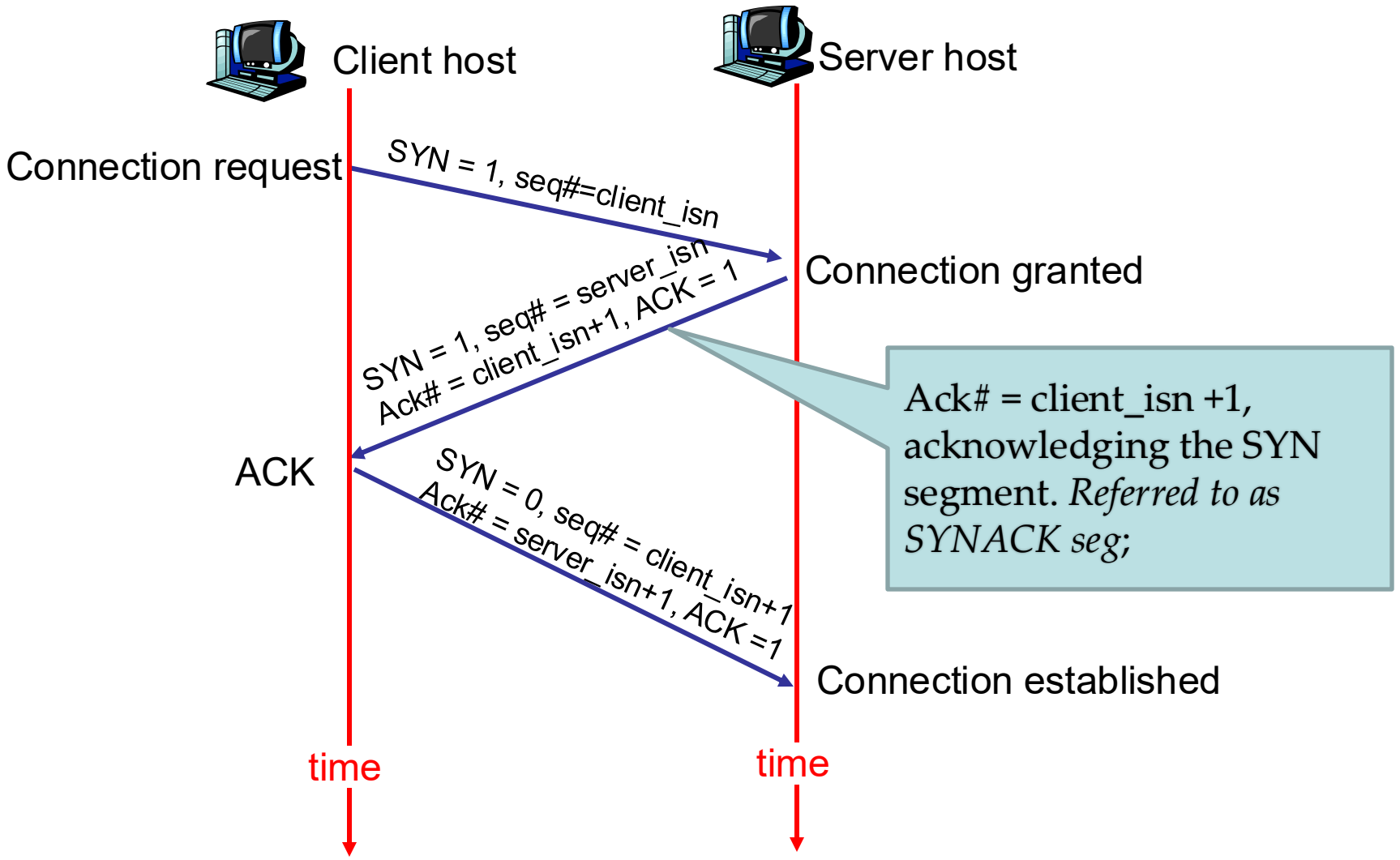
- point-to-point:
  - one sender, one receiver
- reliable, in-order *byte stream:*
  - no (app. layer) “message boundaries”
- pipelined:
  - TCP congestion and flow control set window size
- full duplex data:
  - bi-directional data flow in same connection
  - MSS: maximum segment size
- connection-oriented:
  - handshaking (exchange of control msgs) init’s sender, receiver state before data exchange



TCP segment structure



TCP 3-way handshaking



TCP Connection Management – Three-way Handshake

Step 1: client host sends TCP SYN segment (*SYN bit is set to 1*) to server

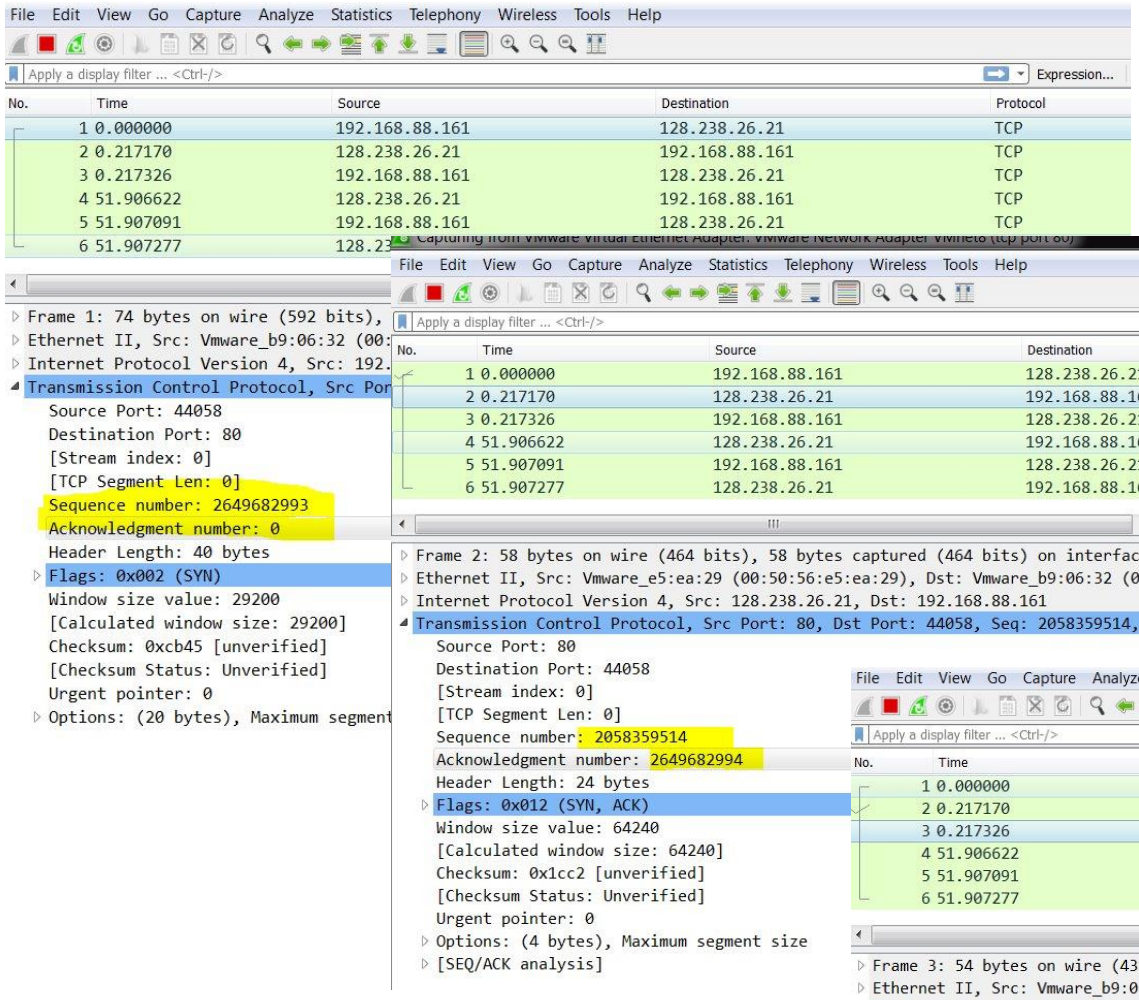
- specifies initial seq # (*randomly chosen*), client_isn;
- no data

Step 2: server host receives SYN, replies with SYNACK segment

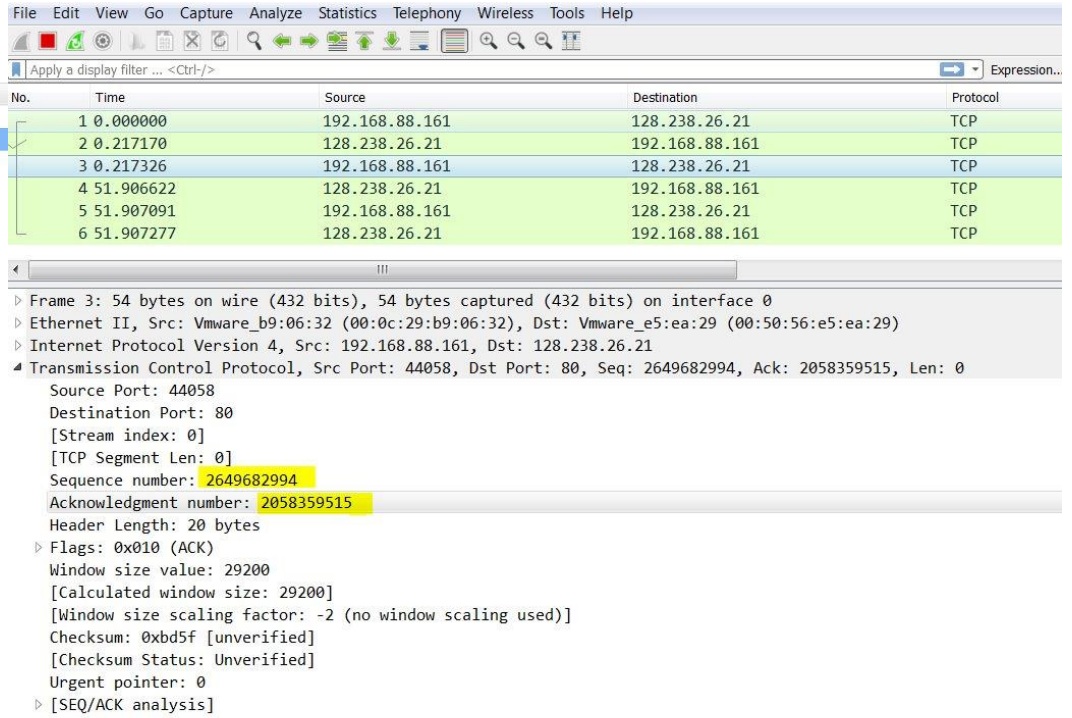
- specifies server initial seq. # (*randomly chosen*), server_isn;
- SYN = 1; ACK# = client_isn +1; ACK = 1; seq# = server_isn;

Step 3: client receives SYNACK; replies with ACK segment, which may contain data;

SYN = 0; ACK = 1; ACK# = server_isn + 1; seq# = client_isn + 1;


File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
Expression...
No. Time Source Destination Protocol
1 0.000000 192.168.88.161 128.238.26.21 TCP
2 0.217170 128.238.26.21 192.168.88.161 TCP
3 0.217326 192.168.88.161 128.238.26.21 TCP
4 51.906622 128.238.26.21 192.168.88.161 TCP
5 51.907091 192.168.88.161 128.238.26.21 TCP
6 51.907277 128.23
Frame 1: 74 bytes on wire (592 bits),
Ethernet II, Src: Vmware_b9:06:32 (00:
Internet Protocol Version 4, Src: 192.
Transmission Control Protocol, Src Por
Source Port: 44058
Destination Port: 80
[Stream index: 0]
[TCP Segment Len: 0]
Sequence number: 2649682993
Acknowledgment number: 0
Header Length: 40 bytes
Flags: 0x002 (SYN)
Window size value: 29200
[Calculated window size: 29200]
Checksum: 0xcb45 [unverified]
[Checksum Status: Unverified]
Urgent pointer: 0
Options: (20 bytes), Maximum segment

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
Expression...
No. Time Source Destination Protocol
1 0.000000 192.168.88.161 128.238.26.21 TCP
2 0.217170 128.238.26.21 192.168.88.161 TCP
3 0.217326 192.168.88.161 128.238.26.21 TCP
4 51.906622 128.238.26.21 192.168.88.161 TCP
5 51.907091 192.168.88.161 128.238.26.21 TCP
6 51.907277 128.238.26.21 192.168.88.161 TCP
Frame 2: 58 bytes on wire (464 bits), 58 bytes captured (464 bits) on interface 0
Ethernet II, Src: Vmware_e5:ea:29 (00:50:56:e5:ea:29), Dst: Vmware_b9:06:32 (00:0c:29:b9:06:32)
Internet Protocol Version 4, Src: 128.238.26.21, Dst: 192.168.88.161
Transmission Control Protocol, Src Port: 80, Dst Port: 44058, Seq: 2058359514, Ack: 2649682994, Len: 0
Source Port: 80
Destination Port: 44058
[Stream index: 0]
[TCP Segment Len: 0]
Sequence number: 2058359514
Acknowledgment number: 2649682994
Header Length: 24 bytes
Flags: 0x012 (SYN, ACK)
Window size value: 64240
[Calculated window size: 64240]
Checksum: 0x1cc2 [unverified]
[Checksum Status: Unverified]
Urgent pointer: 0
Options: (4 bytes), Maximum segment size
[SEQ/ACK analysis]


File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
Expression...
No. Time Source Destination Protocol
1 0.000000 192.168.88.161 128.238.26.21 TCP
2 0.217170 128.238.26.21 192.168.88.161 TCP
3 0.217326 192.168.88.161 128.238.26.21 TCP
4 51.906622 128.238.26.21 192.168.88.161 TCP
5 51.907091 192.168.88.161 128.238.26.21 TCP
6 51.907277 128.238.26.21 192.168.88.161 TCP
Frame 3: 54 bytes on wire (432 bits), 54 bytes captured (432 bits) on interface 0
Ethernet II, Src: Vmware_b9:06:32 (00:0c:29:b9:06:32), Dst: Vmware_e5:ea:29 (00:50:56:e5:ea:29)
Internet Protocol Version 4, Src: 192.168.88.161, Dst: 128.238.26.21
Transmission Control Protocol, Src Port: 44058, Dst Port: 80, Seq: 2649682994, Ack: 2058359515, Len: 0
Source Port: 44058
Destination Port: 80
[Stream index: 0]
[TCP Segment Len: 0]
Sequence number: 2649682994
Acknowledgment number: 2058359515
Header Length: 20 bytes
Flags: 0x010 (ACK)
Window size value: 29200
[Calculated window size: 29200]
[Window size scaling factor: -2 (no window scaling used)]
Checksum: 0xbd5f [unverified]
[Checksum Status: Unverified]
Urgent pointer: 0
[SEQ/ACK analysis]

TCP Connection Management – Close a Connection

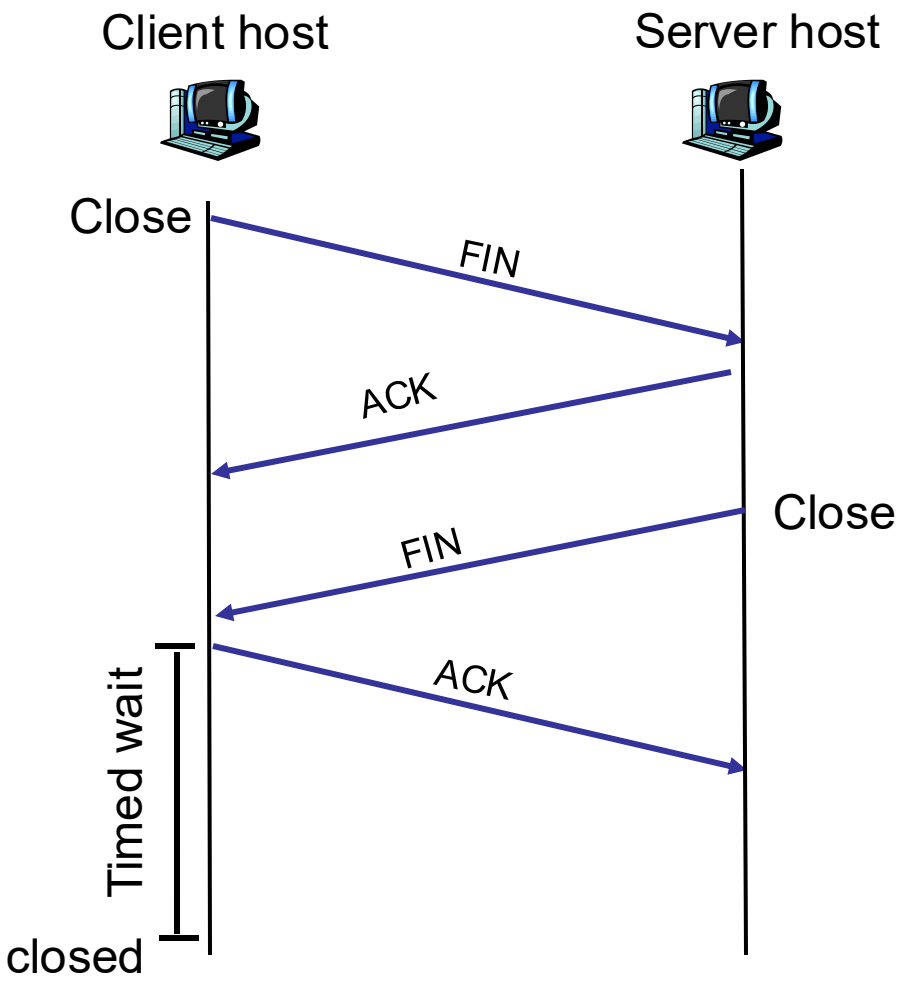
Normal closing:

Step 1: the client sends a TCP FIN control segment to the server.

Step 2: the server receives the FIN segment and replies with ACK; sends a FIN segment.

Step 3: the client receives the FIN segment and replies with ACK.

Step 4: the server receives ACK. Connection closed.



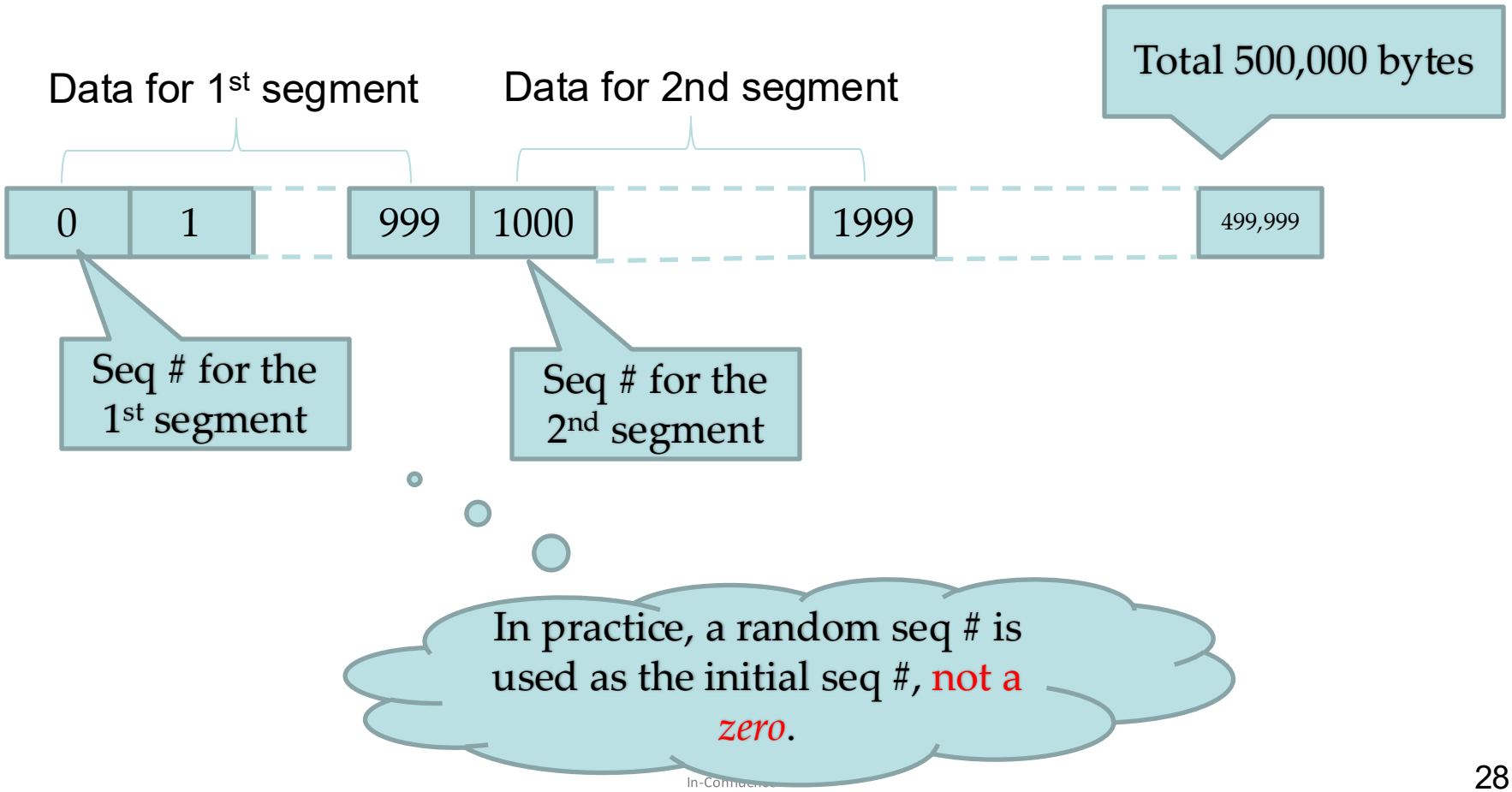
TCP Reliable Data Transfer: An Overview

- TCP creates RDT service on top of IP's unreliable service
- Pipelined segments
- Cumulative ACKs
- TCP uses single retransmission timer
- Retransmissions are triggered by:
  - timeout events
  - duplicate ACKs (*fast retransmit*)

TCP Segments and Sequence Numbers

Seq. #'s:

- usually the byte-stream "number" of the first byte in segment's data

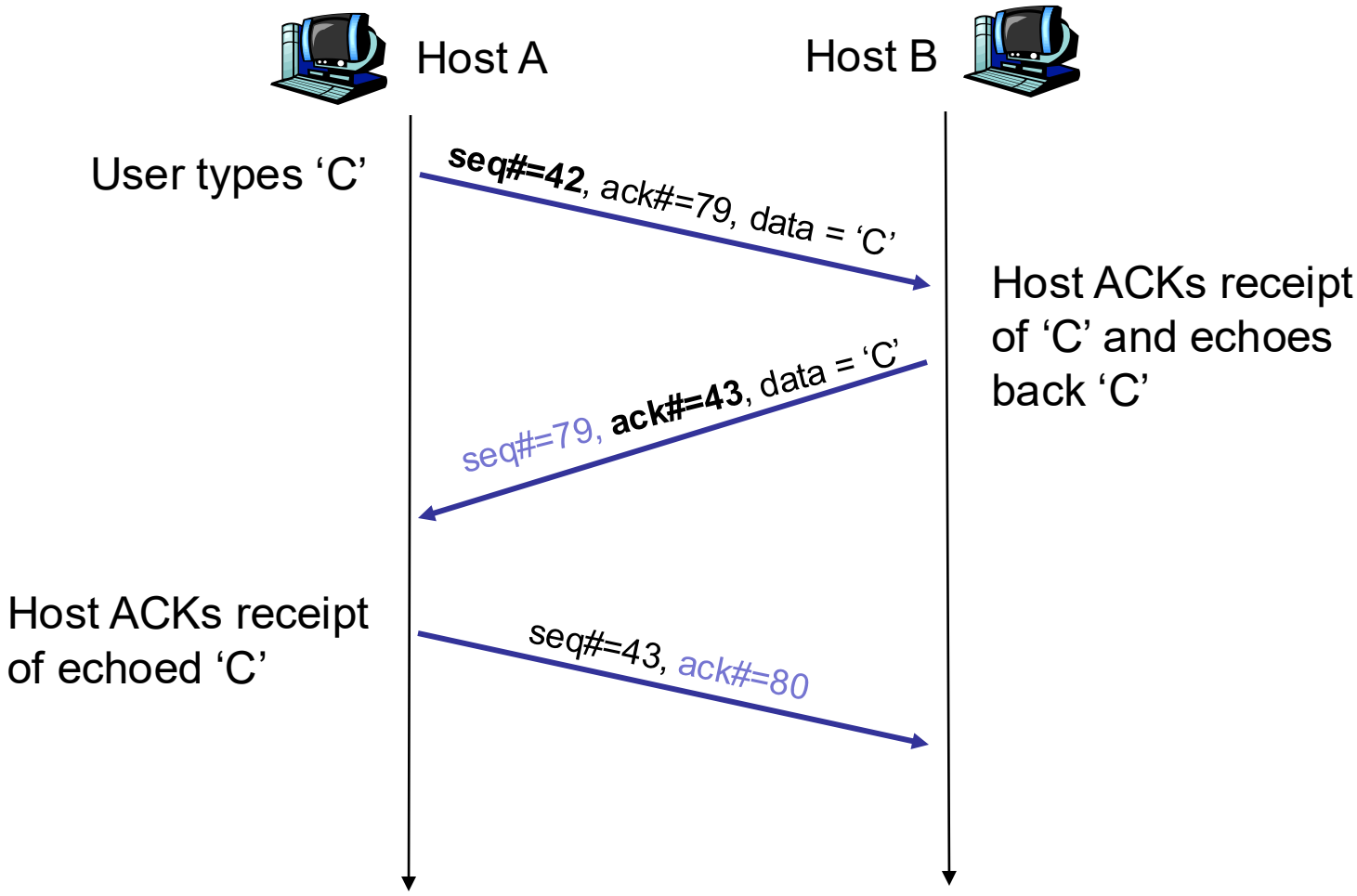


TCP ACK Number

ACKs:

- seq # of next byte expected from the other side
- ACK(seq #) means "*all bytes up to (seq# -1)* are received correctly"
- Cumulative ACK (like Go-Back-N)

An Example



Sequence and acknowledgment numbers for a simple Telnet application over TCP

TCP RDT Sender Events:

data rcvd from app:

- Create segment with seq #
- start timer if not already running

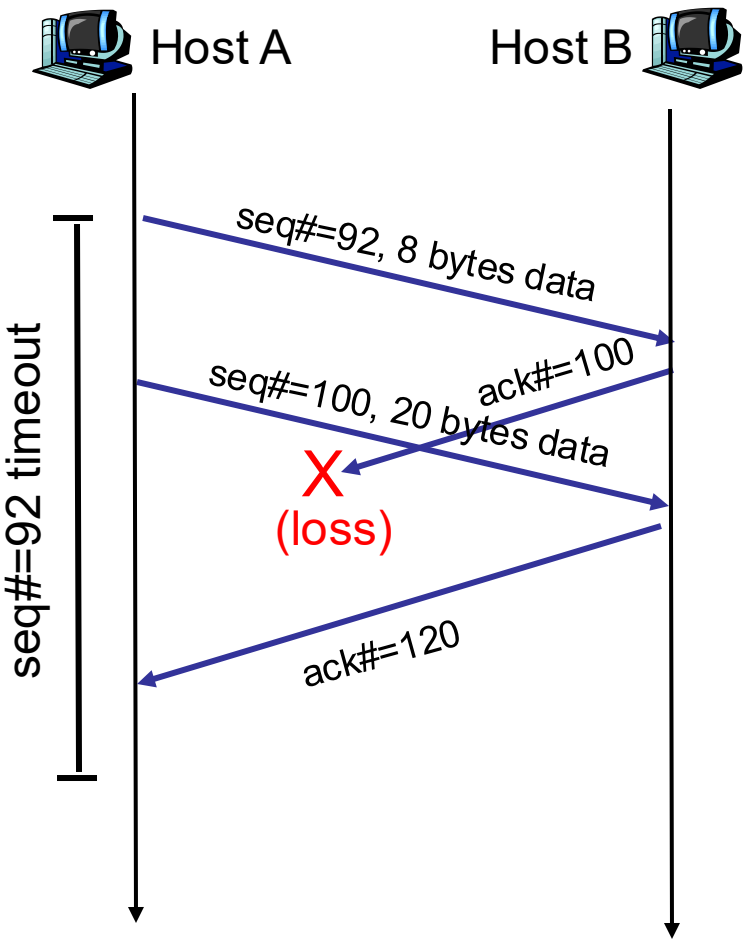
timeout:

- retransmit segment with the *smallest* seq#;
- restart timer

Ack rcvd:

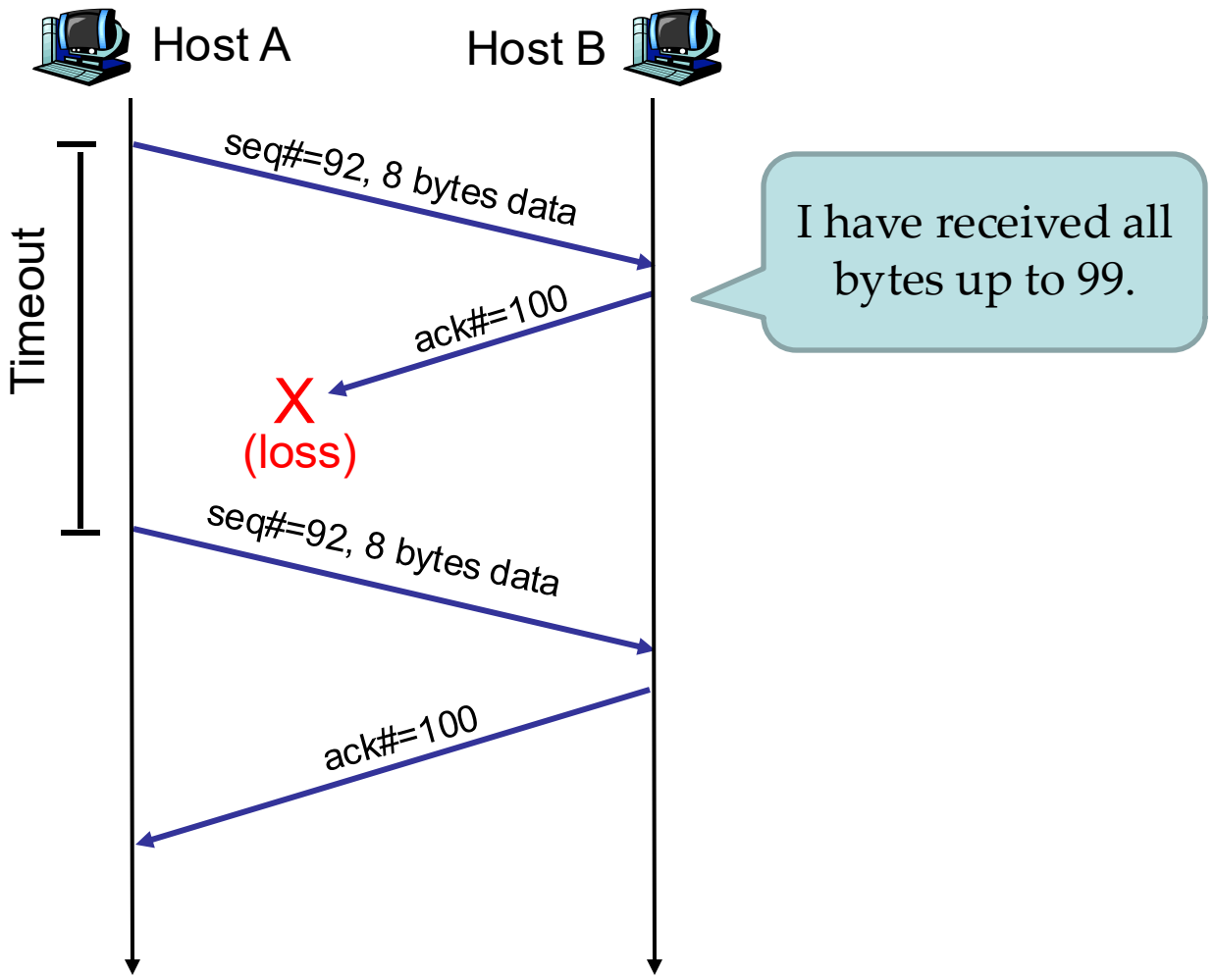
- Cumulative ACK;
- Slide window;
- … (later);

TCP RDT – Cumulative ACK



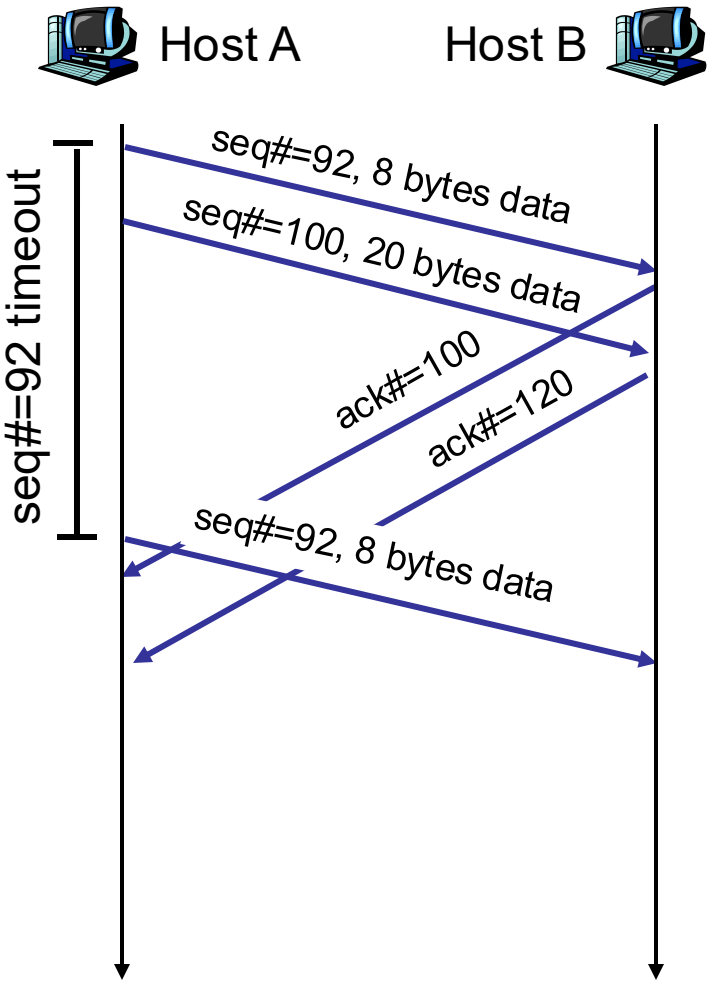
Cumulative ACK: all bytes up to (120-1) are received!

TCP RDT - Timeout



Retransmission due to a lost acknowledgment

TCP RDT – Timeout (cont.)

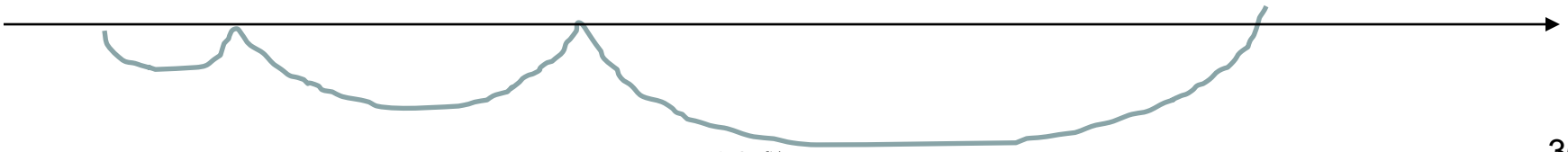


Retransmission of one segment only.
(different from Go-Back-N)

Some Refinement

timeout:

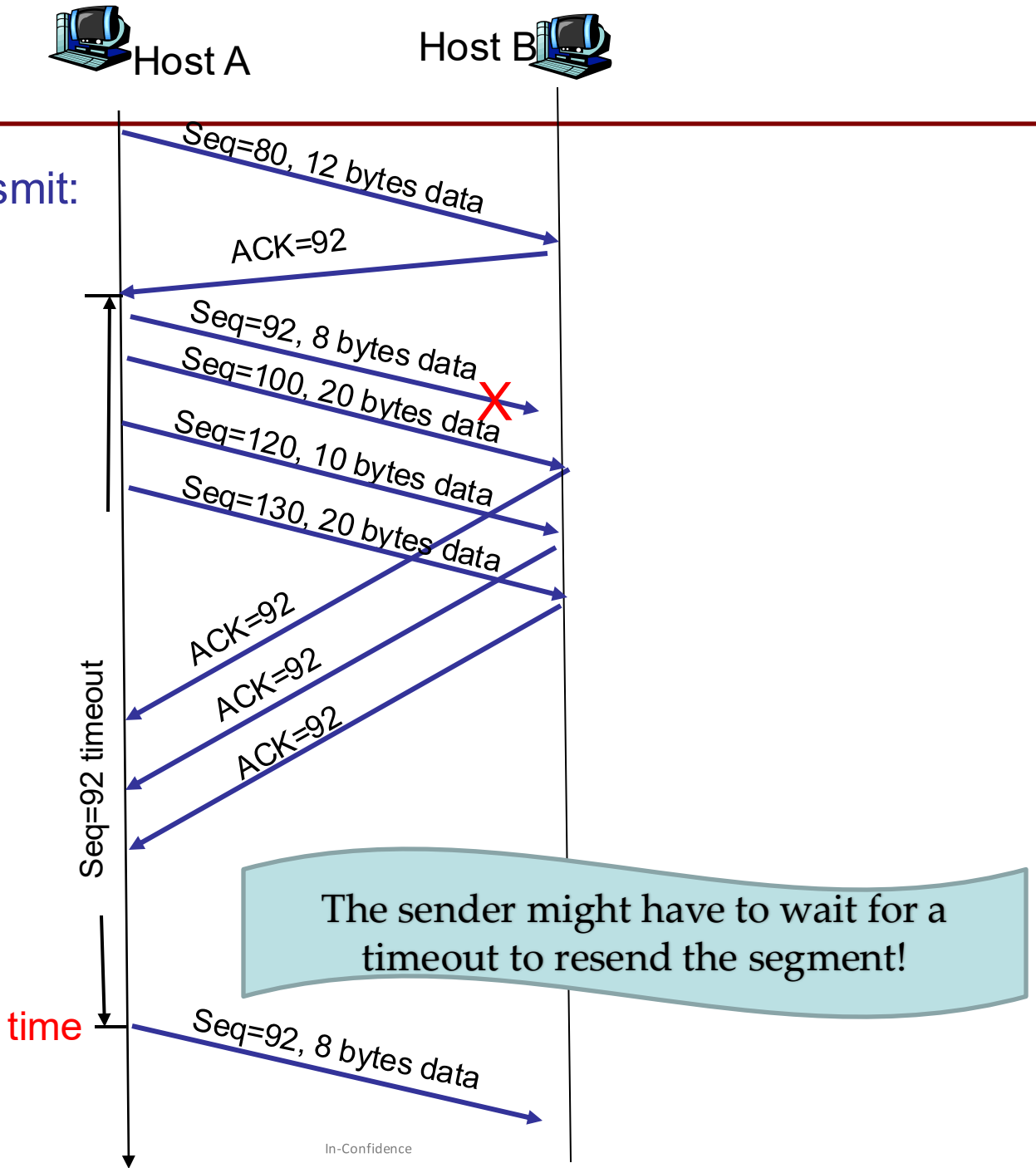
- When timeout expires after `TimoutInterval;`
  - Retransmits the unAck'd segment with smallest seq#; (*not all of the unAck'd pkts, different from Go-Back-N*)
  - (new) `TimoutInterval` =2* (previous) `TimeoutInterval`
- Motivation: no packet was received for the last timeout period
  - Could be a congestion… *trying to slow down the sending process*;
- `TimoutInterval` uses *initial value (calculated based on most recent EstimatedRTT and DevRTT, to be discussed later)* when timer is started in events "data rcvd from app" or "Ack rcvd" is activated;



Fast Retransmit

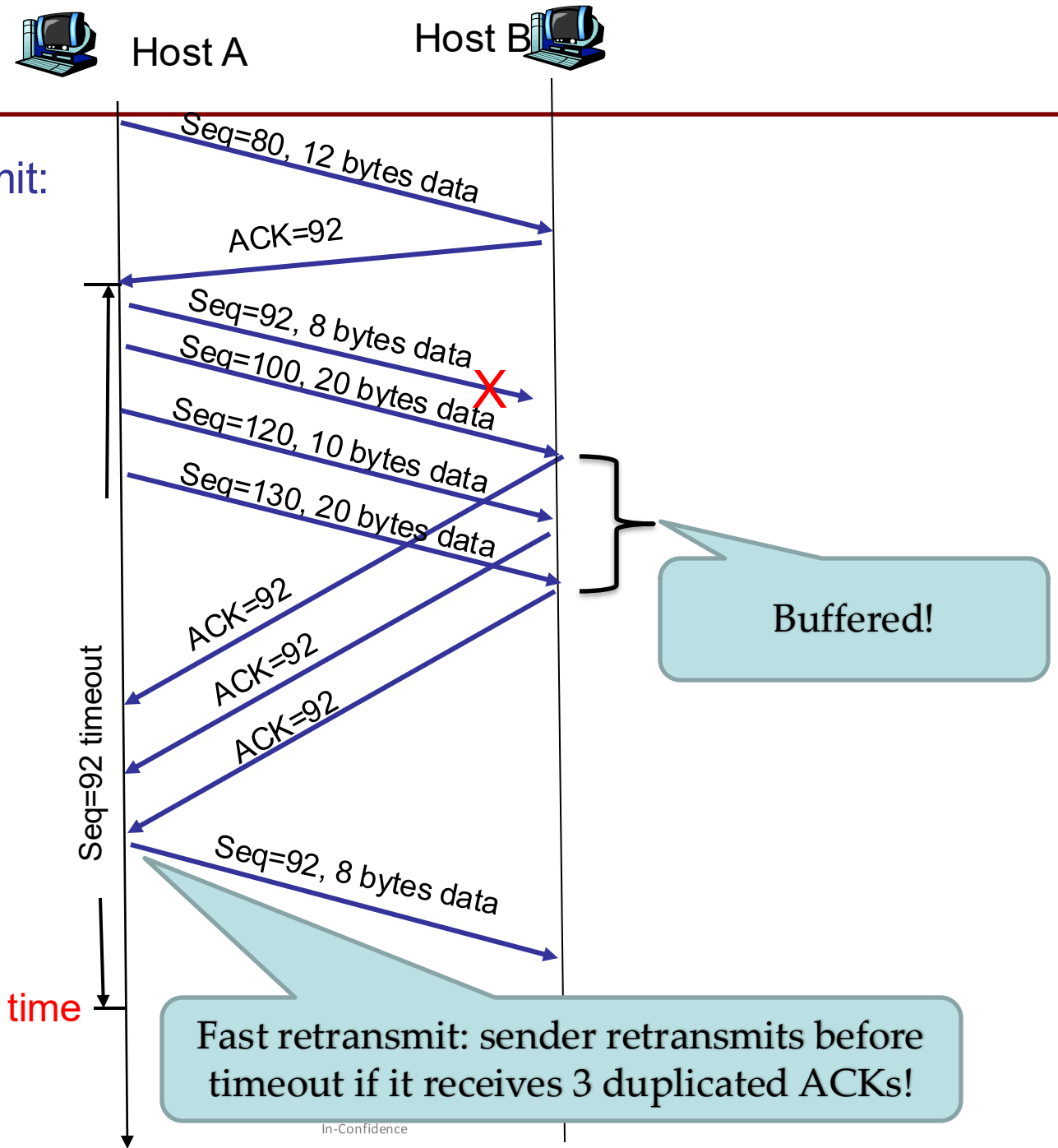
- If sender receives 3 duplicated ACKs for the same data, it supposes that segment after the ACK'd data was lost:
  - fast retransmit: resend segment before timer expires
- Time-out period is often relatively long:
  - long delay before resending lost packet
- Detect lost segments via duplicate ACKs.
  - Sender often sends many segments back-to-back
  - If segment is lost, there will likely be many duplicate ACKs.

Without fast retransmit:



The sender might have to wait for a timeout to resend the segment!

With fast retransmit:



TCP Reliable Data Transfer

Go-Back-N or Selective Repeat/Reject?

- Cumulative ACKs (similar to GBN)
- Many TCP implementations buffer correctly received but out-of-order segments (similar to SR)
- When there is timeout, sender retransmits the unAck'd segment with the smallest seq# only

TCP Reliable Data Transfer

- There are further recommendations for ACK generation at the TCP receiver side.
  - To avoid sending two many ACKs
  - RFC 1122, RFC 2581
- TCP can use selective ACK options [RFC 2018].

How Long Should the Sender Wait?

TCP Round-Trip Time and Timeout

- If the timeout value is too small, the sender will time out too fast resulting in unnecessary retransmissions.
- If the timeout value is too large, the sender will take too long to recover from errors (i.e. excessive delays when packets are lost).
- TCP sets timeout as a function of the RTT (Round-Trip Time).

TCP Round Trip Time and Timeout

Q: how to estimate RTT?

- **SampleRTT**: measured time from segment transmission until ACK receipt
  - ignore retransmissions (*do not sample retransmitted segments*);
- **SampleRTT** will vary, want estimated RTT "typical"
  - average several recent measurements

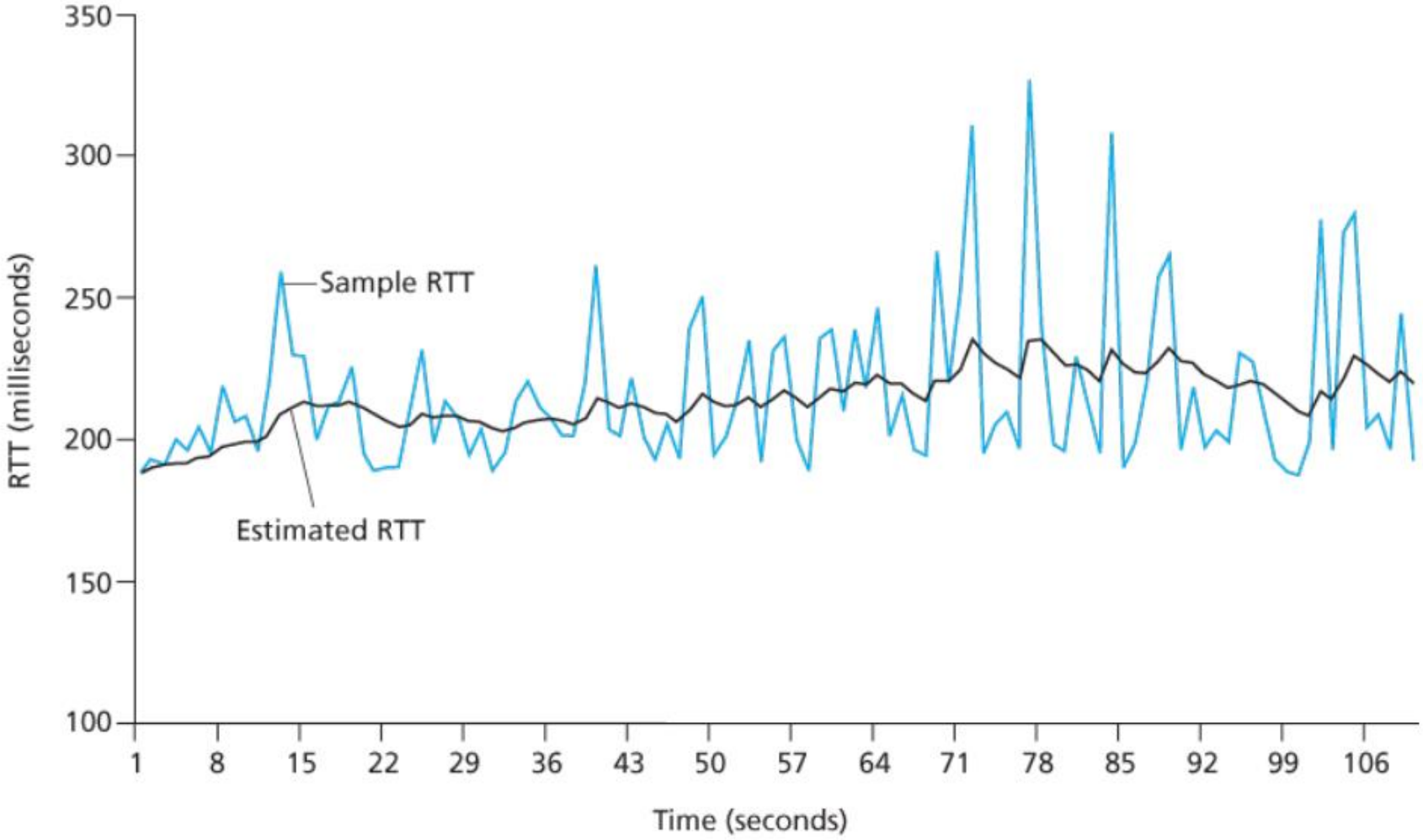
TCP Round-Trip Time and Timeout (cont.)

$$\texttt{EstimatedRTT} = (1 - \alpha) * \texttt{EstimatedRTT} + \alpha * \texttt{SampleRTT}$$

- Exponential weighted moving average (EWMA)
  - The weighted average puts more weight on recent samples which better reflect the current congestion in the network
  - The weight of a given SampleRTT decays exponentially fast as the updates proceed.

- Typical value: $\alpha = 0.125$

TCP Round-Trip Time and Timeout (cont.)



Source: James F. Kurose & Keith W. Ross, Computer networking: a top-down approach featuring the Internet, 3rd edition, 2005, Figure 3.32.

TCP Round-Trip Time and Timeout (cont.)

Setting the timeout

- Measure the variability of the RTT as an estimate of how much SampleRTT deviates from EstimatedRTT

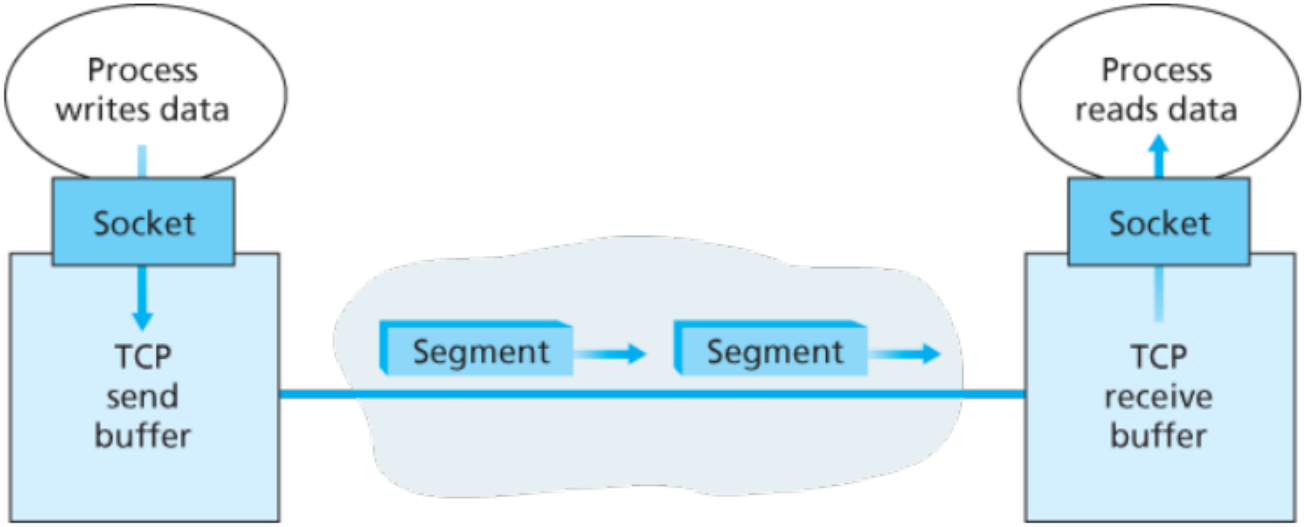
```
DevRTT = (1 - β) * DevRTT +
                β * |SampleRTT-EstimatedRTT|

(typically, β = 0.25)
```

```
TimeoutInterval = EstimatedRTT + 4 * DevRTT
```

TCP Flow Control and Congestion Control

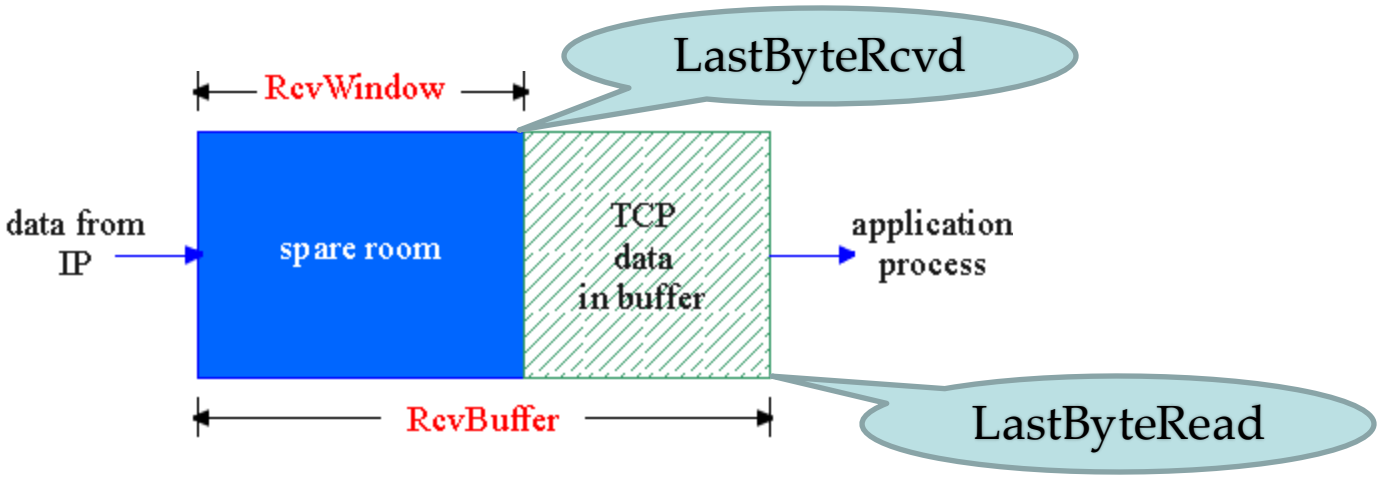
TCP Flow Control

- Flow control is a speed-matching service
  - Match the rate at which the sender is sending against the rate at which the receiver is reading.



Source: James F. Kurose & Keith W. Ross, Computer networking: a top-down approach featuring the Internet, 3rd edition, 2005, Figure 3.28.

TCP Flow Control: How It Works

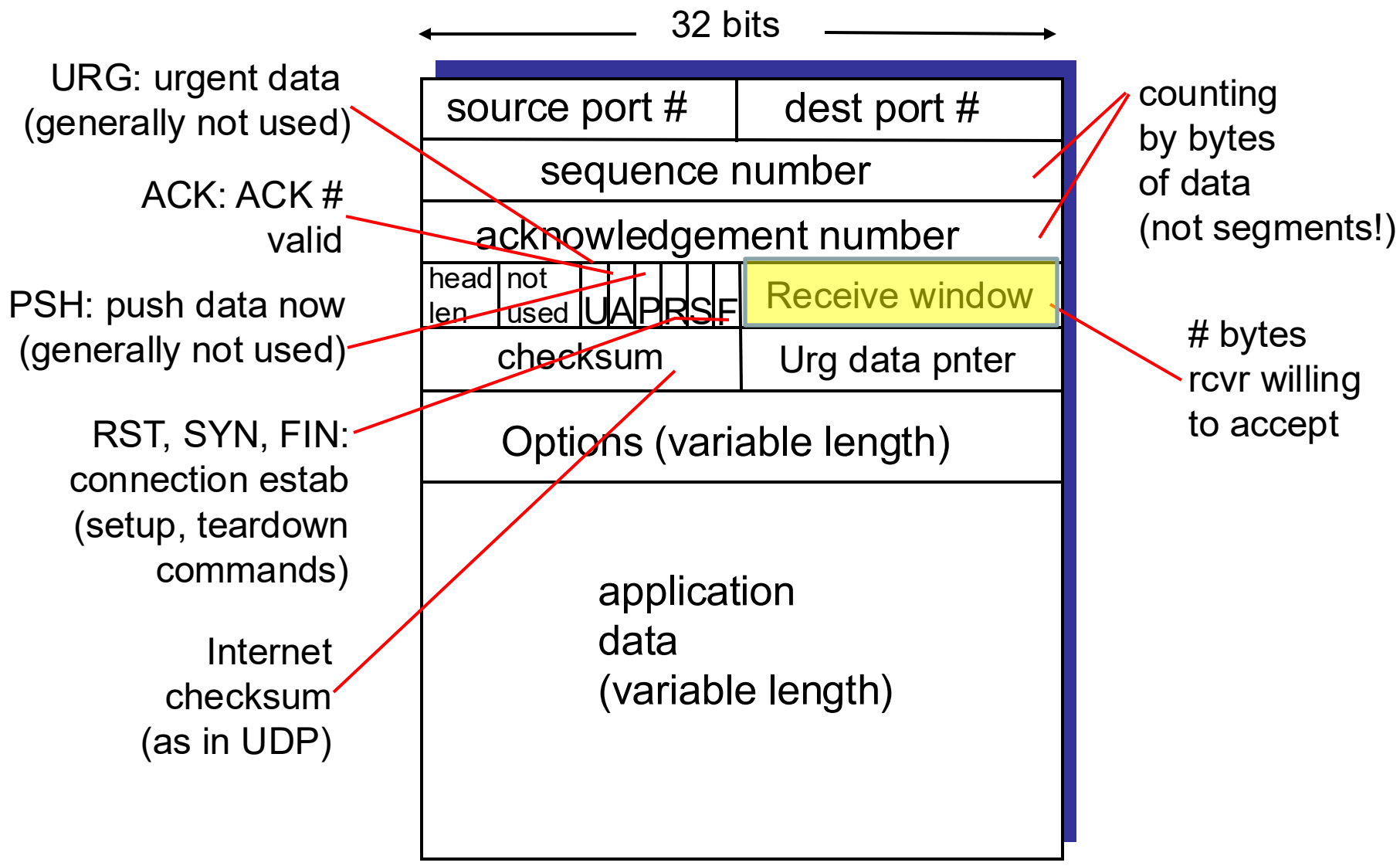


Spare room in buffer = RcvWindow

= RcvBuffer - [LastByteRcvd - LastByteRead]

Question: How does the connection use the variable RcvWindow to provide the flow control service?

TCP Segment Structure

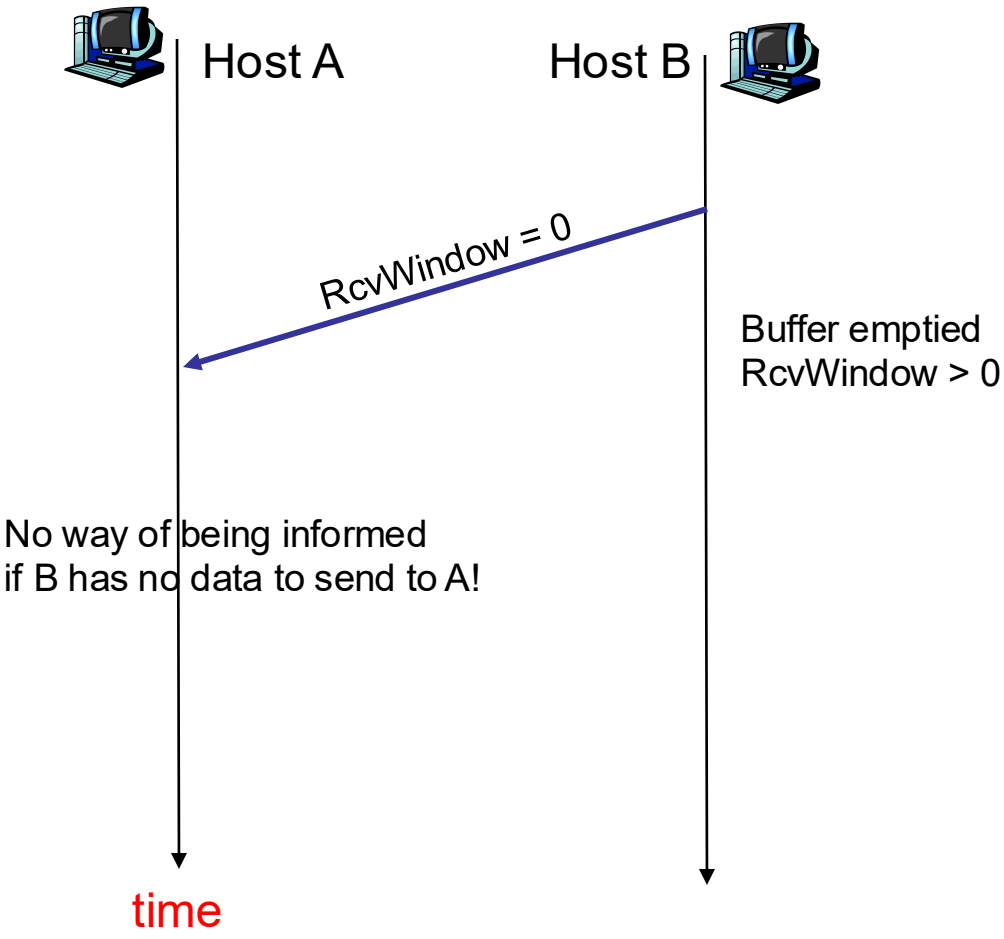


TCP Flow Control: How It Works (cont.)

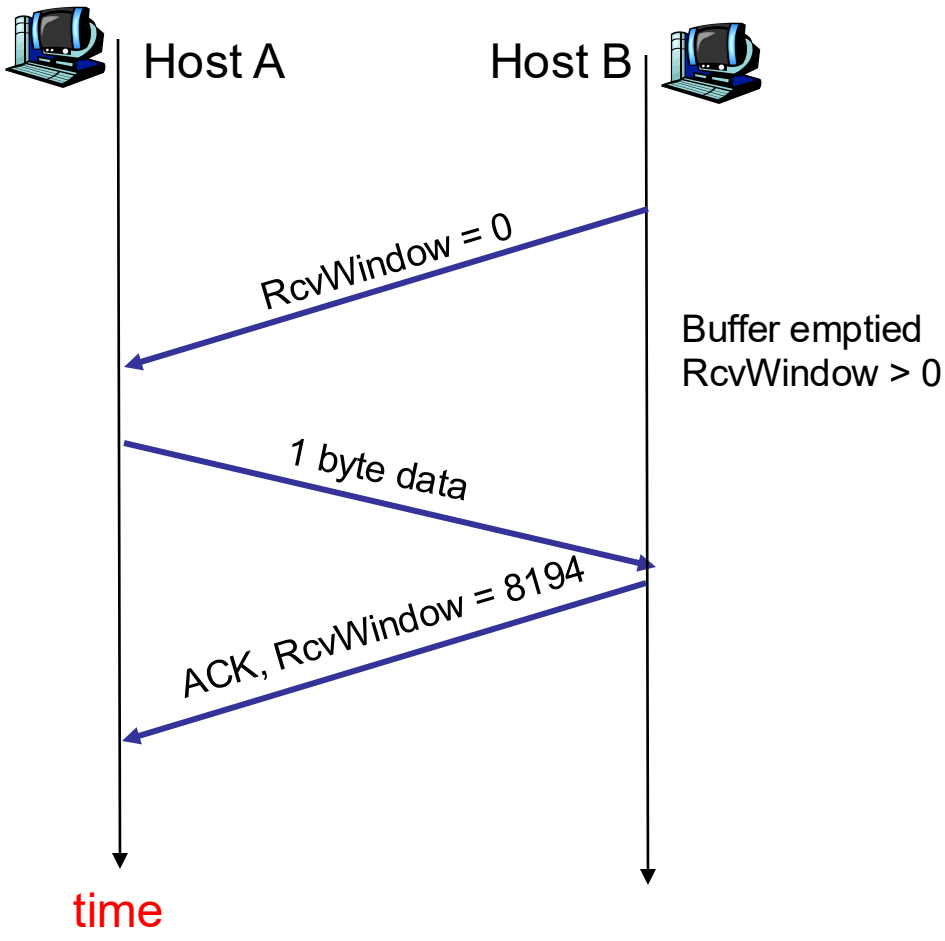
- Suppose that host A is sending a large file to host B over a TCP connection.

- Host B places its current value of RcvWindow in the window field of every segment it sends to A.
  - Initially, host B sets RcvWindow = RcvBuffer.
  - Host A keeps track of two variables, LastByteSent and LastByteAcked, and makes sure that LastByteSent - LastByteAcked <= RcvWindow.

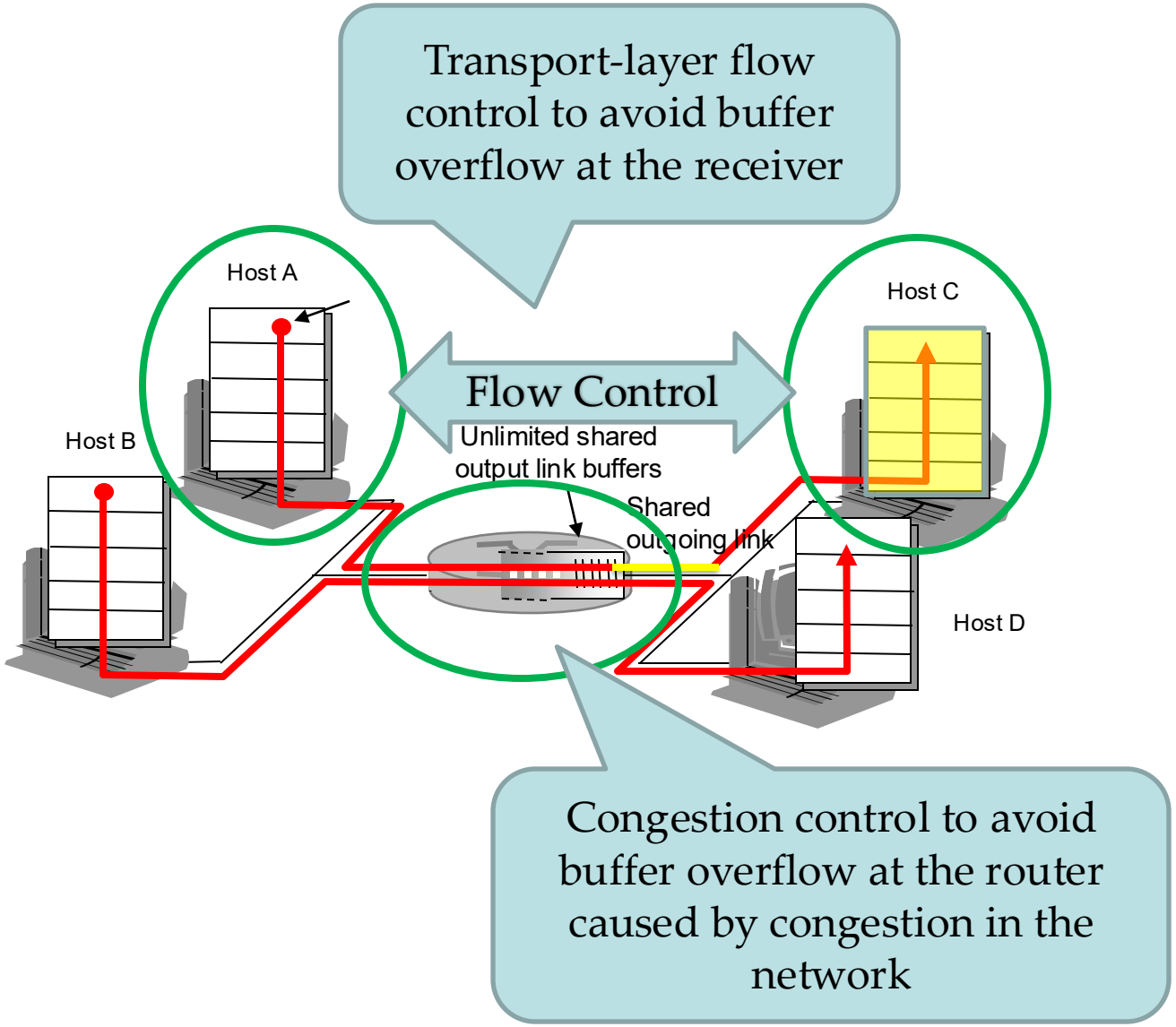
Pathological Scenario



Solution – Continue to Send Small Segments to B



Flow Control vs Congestion Control



Principles of Congestion Control

Congestion

- Informally: “too many sources attempting to send data at too high a rate”

- Manifestations (symptoms)
  - lost packets (buffer overflow at routers)
  - long delays (queueing in router buffers)

Approaches to Congestion Control

Network-assisted congestion control

- Routers provide feedback to the end systems, such as a single bit indicating congestion at a link, the explicit transmission rate supported on an outgoing link

End-to-end congestion control

- No explicit support from the network layer
- Congestion inferred by the end-systems based on observed packet loss and delay

TCP Congestion Control

- Use end-to-end congestion control (no network assistance)

- Keep track of the congestion window that limits the sending rate

$$\texttt{LastByteSent-LastByteAcked} \leq \min\{\texttt{CongWin}, \texttt{RcvWindow}\}$$

TCP Congestion Control (cont.)

- Roughly at the beginning of every RTT, the constraint permits the sender to send at most **CongWin** bytes.

- By adjusting **CongWin** according to network congestion, the sender can adjust its rate at which it sends data into its connection.

$$\text{Sending rate} = \frac{\text{CongWin}}{\text{RTT}} \text{ bytes/sec}$$

TCP Congestion Control (cont.)

How does a TCP sender detect congestion?

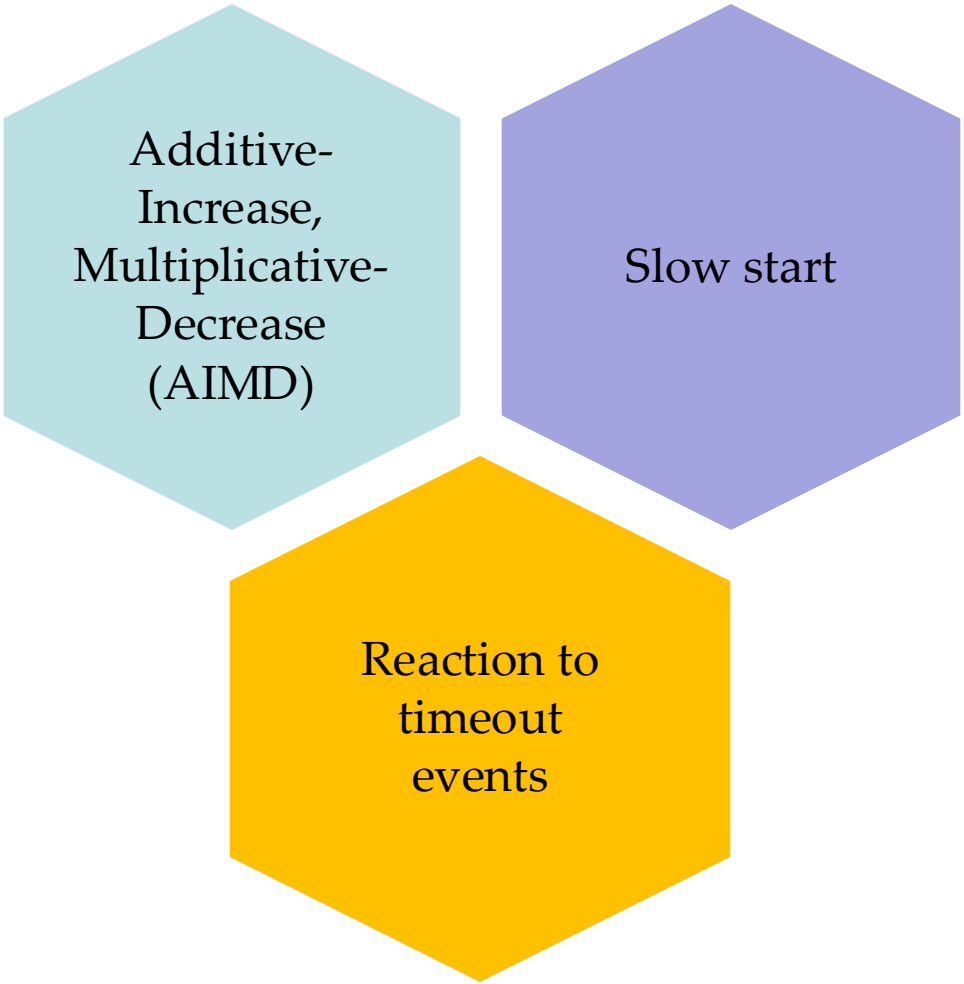
- **Loss event** = timeout or 3 duplicate ACKs
- The TCP sender reduces the send rate after a loss event

TCP Congestion Control (cont.)

How does a TCP sender adjust CongWin?

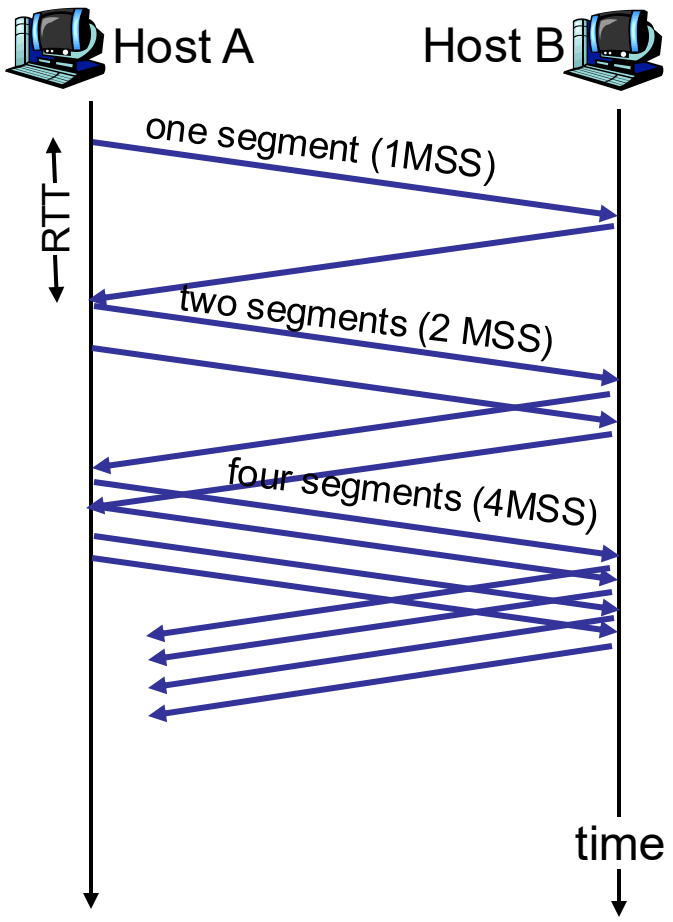
- ACKs indicate a congestion-free path and loss events indicate a congested path
  - If ACKs (for previously unacknowledged segments) arrive, increase **CongWin**
  - If there is a timeout or duplicate ACKs, decrease **CongWin**

The celebrated TCP Congestion-Control Algorithm



TCP Slow Start

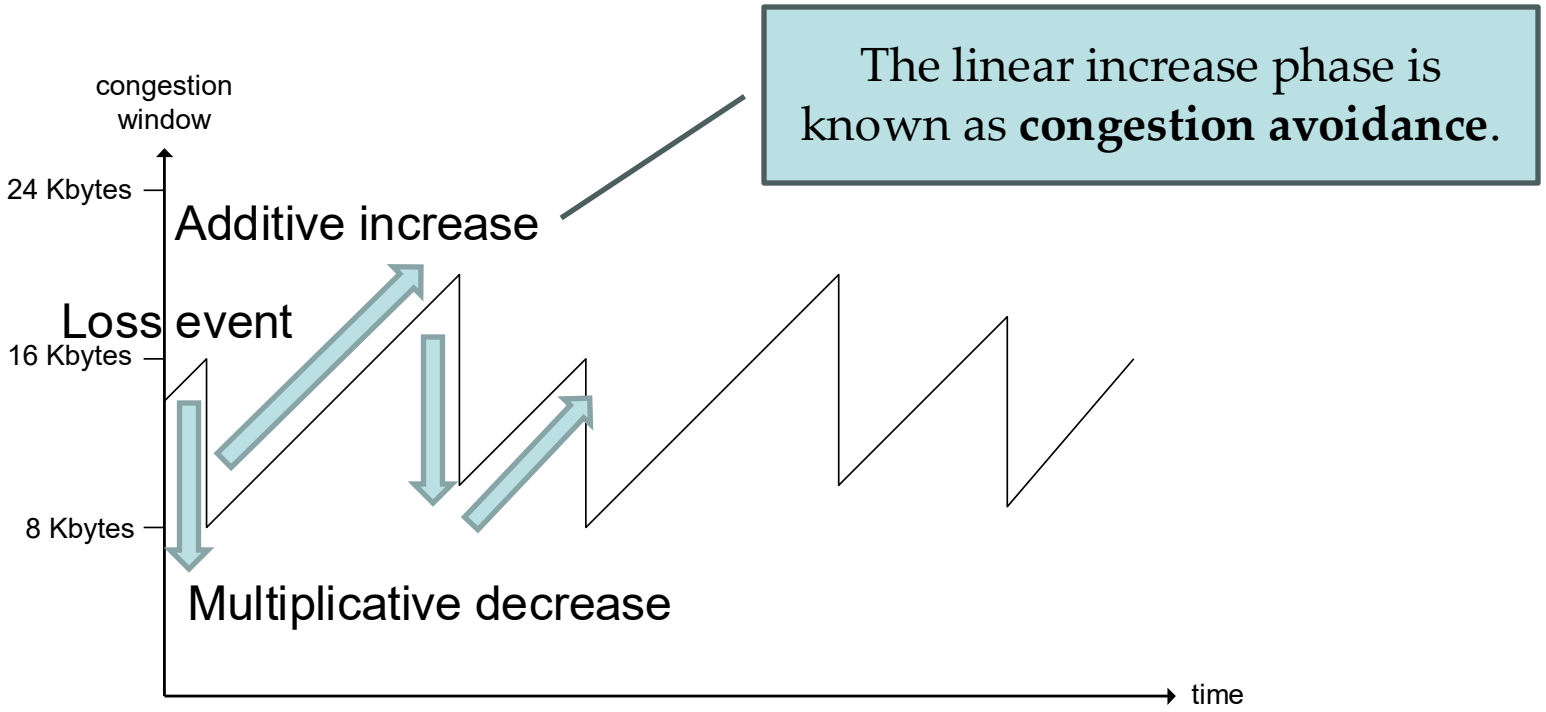
- When a connection begins
  - **CongWin** = 1 MSS (Maximum Segment Size)
  - Initial sending rate = MSS/RTT
- Double **CongWin** every RTT
  - **CongWin** increases by 1 MSS every time a transmitted segment is first acknowledged
- Summary: the TCP sending rate starts slow but grows exponentially during the slow start phase



TCP AIMD (Saw-toothed Pattern)

Additive increase: increase **CongWin** by 1 MSS **every RTT**

Multiplicative decrease: cut **CongWin** in half when a loss event occurs



Source: James F. Kurose & Keith W. Ross, Computer networking: a top-down approach featuring the Internet, 3rd edition, 2005, Figure 3.50.

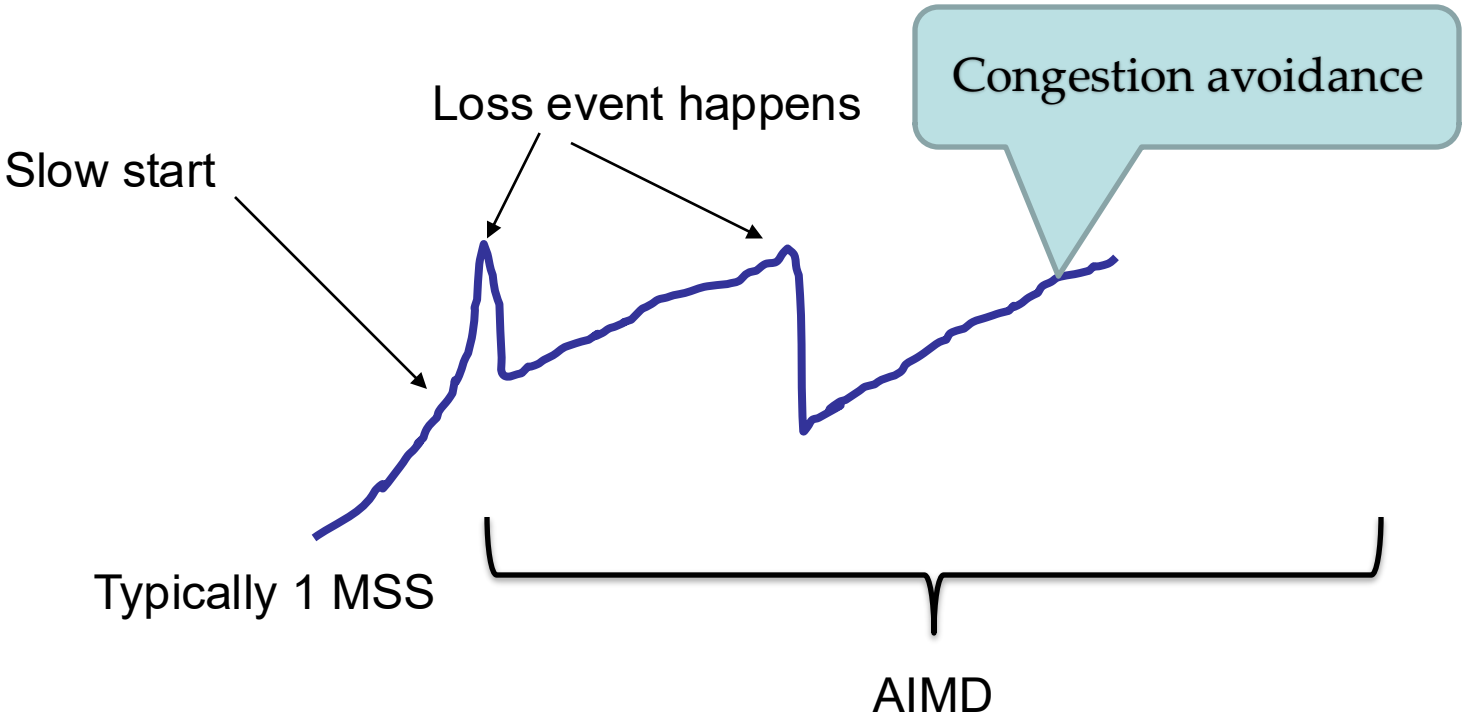
TCP AIMD

- Increase CongWin by a single MSS every RTT

An example

- Initial CongWin = 1 MSS
- After sending one segment and receiving one ACK within one RTT, CongWin = 1 MSS + 1 MSS = 2 MSS
- After sending two segments and receiving two ACKs within one RTT, CongWin = 2 MSS+ 1 MSS = 3 MSS
- Continue, CongWin = 3 MSS+ 1 MSS = 4 MSS

Tricks so far



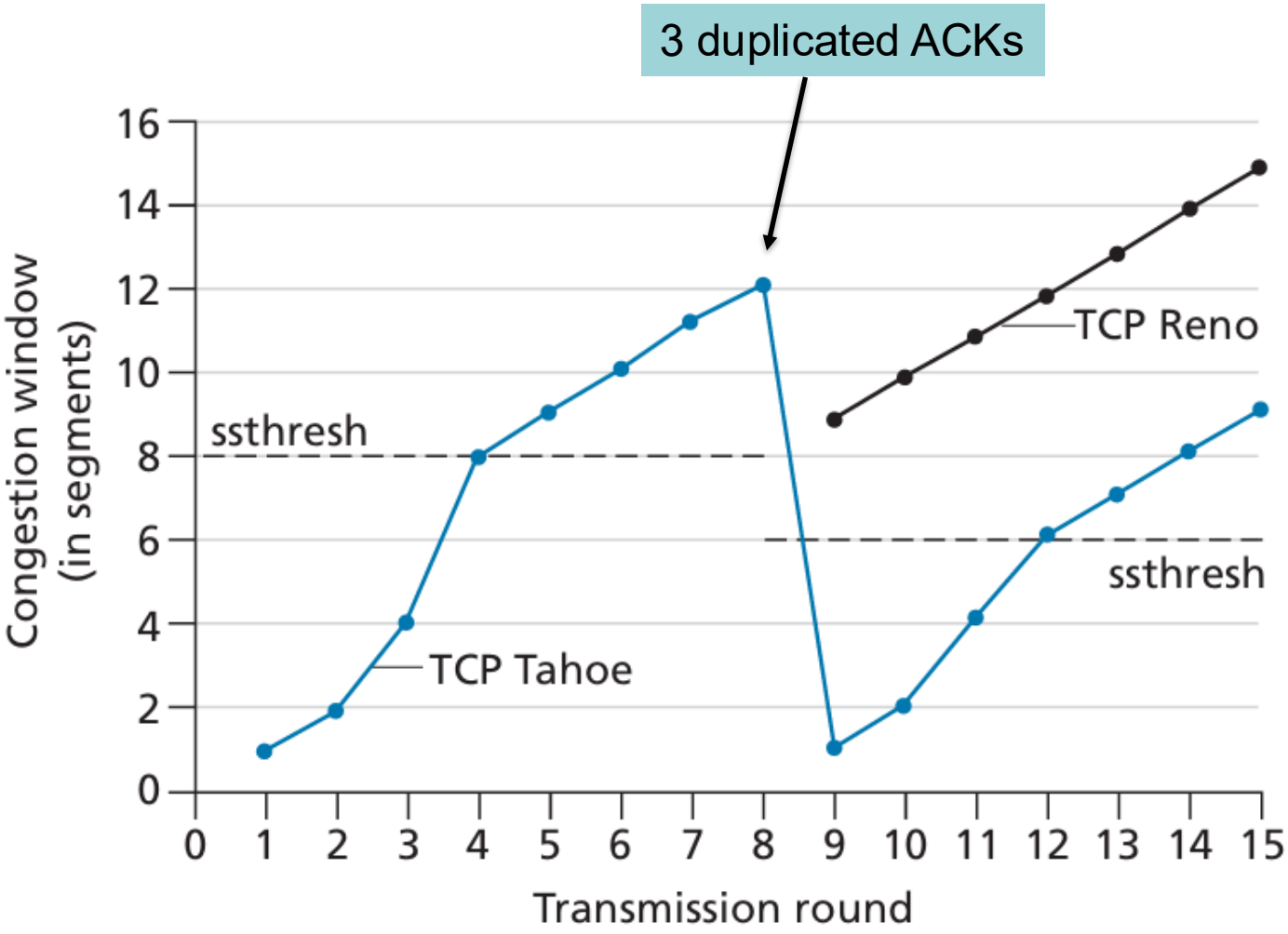
TCP has some refined actions towards a loss event, depending on whether it is a timeout or 3 dup-ACKs.

- 3 dup ACKs indicates network is still capable of delivering some segments
- timeout is "more alarming";

Refinement – Reaction to Timeout

- After 3 dup ACKs:
  - **`ssthresh`** = **`CongWin/2`**
  - **`CongWin`** = **`CongWin/2 + 3 MSS`**
  - window then grows linearly for each additional duplicate ACK
  - **`CongWin`** = **`ssthresh`** if a new ACK arrives and window grows linearly again.
- But after timeout event:
  - **`CongWin`** is set to 1 MSS;
  - **`ssthresh`** = **`CongWin/2`**
  - window then grows exponentially to a threshold (**`ssthresh`**), then grows linearly

TCP Tahoe vs TCP Reno



Source: James F. Kurose & Keith W. Ross, Computer networking: a top-down approach featuring the Internet, 8th edition, 2021, Figure 3.52.

References

- James F. Kurose & Keith W. Ross, Computer networking: a top-down approach featuring the Internet, Addison Wesley, 3rd edition, 2005.
  - Chapter 3 Transport Layer (3.1 – 3.3, 3.5 – 3.7)
- William Stallings, Data and Computer Communications, Pearson, 10th edition, 2013.
  - Chapter 15 Transport Protocols
- Andrew S. Tanenbaum & David J. Wetherall, Computer Networks, Prentice Hall, 5th edition, 2010.
  - Chapter 6 The Transport Layer (6.2, 6.3, 6.5)

Acknowledgements

- Lecture notes are developed based on slides from the following three sources:
  - Dr Mengmeng Ge's lecture notes for COSC264, University of Canterbury, 2024
  - Assoc Prof Dan Kim's lecture notes for COSC264, University of Canterbury, 2017
  - Prof Aleksandar Kuzmanovic's lecture notes for CS340, Northwestern University, 2005, https://users.cs.northwestern.edu/~akuzma/classes/CS340-w05/lecture_notes.htm